--- Working Iteration 1: has export, list all, list filtered, no main menu

```
#include <iostream>
#include <algorithm>
#include <fstream>
#include <string>
#include <vector>
#include <array>

using namespace std;
// ____________________________________ INVENTORY
____________________________________
class stockData {
    string id, brand, age, date, status;
    public:
        stockData (string &id, string &brand, string &age, string &date,
string &status) : id(id), brand(brand), age(age), date(date),
status(status){}

        string getID() const { return id; } // Force returned value to
be read-only
        string getBrand() const { return brand; }
        string getAge() const { return age; }
        string getDate() const { return date; }
        string getStatus() const { return status; }
        array<string, 5> getAll() const {return {id, brand, age, date,
status}; }

};

vector<stockData> inventory;
array<int,5> spaceCount = {10, 9, 3, 10, 10};

void extractDataCSV() {
    ifstream motorPHData;
    string line{}, extract{};
    array<string, 5> extVal{};
    motorPHData.open("Copy of MotorPH Inventory Data - March 2023
Inventory Data.csv");
    getline(motorPHData, line);
    getline(motorPHData, line);

    while (getline(motorPHData, line)) {
        for (size_t i = 0; i < extVal.size(); ++i) {
            size_t locSep = line.find(',');
```

```
            // if locSep finds a comma (not "no position" or npos) then
value before comma will be set to array,
            // else set the whole line value to the array
            extVal[i] = (locSep != string::npos) ? line.substr(0,
locSep) : line;

            // if locSep returns a "no position" or npos then break the
loop
            if (locSep == string::npos) {
                break;
            }

            line = line.substr(locSep + 1);
        }

            inventory.push_back(stockData(extVal[3], extVal[2],
extVal[1], extVal[0], extVal[4]));
    }
}

// _____________________________________ INVENTORY COMMANDS
_____________________________________


void writeAndCenter (array<string, 5> &temp) {
    for (size_t i = 0; i < 5; i++) { // Write and Center
        string currentString = temp[i];
        int spaces = (spaceCount[i]-currentString.length());
        cout << string(size_t(spaces/2), ' ') << currentString
            << string(size_t((spaces/2) + (spaces % 2)), ' ') << " | ";
    }
}

void headerWrite () {
    array<string, 5> border{}, headers = {"ID", "Brand", "Age", "Date",
" Status "};
    for (size_t i = 0; i < 5; ++i) {
        border[i] = string(spaceCount[i],'=');
    }
    cout << "| ";
    writeAndCenter(border);
    cout << "\n| ";
    writeAndCenter(headers);
    cout << "\n| ";
    writeAndCenter(border);
    cout << endl;
```

```
}

void list () { // Display list in console
    for (const auto &item : inventory) {
        cout << "| ";
        array<string, 5> temp = item.getAll();
        writeAndCenter(temp);
        cout << " " << endl;
    }
}

void listAll () { // Command: /list all
    headerWrite();
    list();
}

void listFilterWrite (array<string, 5> temp) { // Command: /list filter
- sub1
    cout << "| ";
    writeAndCenter(temp);
    cout << " " << endl;
}


void listFilterType(const string &filterType, const string &filterValue,
array<string, 5> temp) { // Command: /list filter - sub2
    if ((filterType == "brand" && temp[1] == filterValue) ||
    (filterType == "age" && temp[2] == filterValue) ||
    (filterType == "date" && temp[3] == filterValue) ||
    (filterType == "status" && temp[4] == filterValue)) {
        listFilterWrite(temp);
    }
}

void listFilter(const string &filterType, const string &filterValue) {
//Comand: list filter
    headerWrite();
    for (const auto &item : inventory) {
        listFilterType(filterType, filterValue, item.getAll());
    }
}


bool isValidStockNumber(const string &stockNumber) {
    // Based on the csv, min numbers = 0 & min letters = 4, all length =
10
```

```
    if (stockNumber.length() != 10) return false;
    return any_of(stockNumber.begin(), stockNumber.end(), ::isalpha);
}

int main () {
    extractDataCSV();
    listFilter("brand", "Kymco"); // Test
    system("pause");
    return 0;
}
```

--- Working Iteration 2: made changes to accommodate for unordered_map, cleaned ver of work-it-1

```
#include <iostream>
#include <unordered_map>
#include <algorithm>
#include <fstream> // CSV Extract
#include <sstream>
#include <string>
#include <array>

using namespace std;
// ____________________________________ INVENTORY
____________________________________
class stockData {
    string id, brand, age, date, status;
    public:
        stockData() : id(""), brand(""), age(""), date(""), status("")
{} // Forgot to add a default constructor
        stockData (string &id_, string &brand_, string &age_, string
&date_, string &status_)
        : id(id_), brand(brand_), age(age_), date(date_),
status(status_){}

        // Force returned value to be read-only
        string getID() const { return id; }
        string getBrand() const { return brand; }
        string getAge() const { return age; }
        string getDate() const { return date; }
        string getStatus() const { return status; }
        array<string, 5> getAll() const {return {id, brand, age, date,
status}; }

};

unordered_map<string, stockData> inventory; // Key: ID -> stockData
typedef unordered_multimap<string, stockData*> invMap;
invMap brandIndex, ageIndex, dateIndex, statusIndex; // Key: Category ->
stockData*



void extractDataCSV() {
    ifstream motorPHData("Copy of MotorPH Inventory Data - March 2023
Inventory Data.csv"); // simplified
```

```
    if (!motorPHData.is_open()) { // Check if csv data is accessible
        cerr << "Data Extraction Failed: CSV not opened." << endl;
        exit(1);
    }

    string line{};
    array<string, 5> extVal{};

    getline(motorPHData, line);
    getline(motorPHData, line);

    while (getline(motorPHData, line)) {
        stringstream ss(line);

        for (size_t i = 0; i < extVal.size(); ++i) {
            if (!getline(ss, extVal[i], ',')) break;
            // Per slot of extVal: runs getline(ss, extVal[i], ',')
            // ss until ',' is assigned to extVal[i]
            // if items be only 4, getline cant assign sstream to
extVal[4] and returns false bool (this is news to me)
            // false return triggers break
        }

        // Check if extVal[3] (ID) has a length of 10
        // Check if extVal[0] (Date) has a length of 8, 9, or 10
(x/x/xxxx, (xx/x) or x/xx/xxxx, xx/xx/xxxx)
        if (extVal[3].length() != 10 || (extVal[0].length() != 8 &&
extVal[0].length() != 9 && extVal[0].length() != 10)) {
            cerr << "Value mismatch: Check ID or Date length (ID: 10,
Date: 8/9/10)" << endl;
            exit(1);  // Exit the program with an error code
        }

        stockData newStock(extVal[3], extVal[2], extVal[1], extVal[0],
extVal[4]); // Use constructor
        const string& stockID = extVal[3]; // Get stock id for indexing
        inventory.emplace(stockID, move(newStock));

        // Indexing for fast filtering
        stockData* stockPtr = &inventory[stockID];
        brandIndex.emplace(stockPtr->getBrand(), stockPtr);
        ageIndex.emplace(stockPtr->getAge(), stockPtr);
        dateIndex.emplace(stockPtr->getDate(), stockPtr);
        statusIndex.emplace(stockPtr->getStatus(), stockPtr);
    }
}
```

```
// ____________________________________ INVENTORY COMMANDS
____________________________________

array<int,5> spaceCount = {10, 8, 3, 10, 10}; // Number of spaces to
center

void writeAndCenter (array<string, 5> &temp) {
    for (size_t i = 0; i < 5; i++) { // Write and Center
        string currentString = temp[i];
        int spaces = (spaceCount[i]-currentString.length());
        cout << string(size_t(spaces/2), ' ') << currentString
            << string(size_t((spaces/2) + (spaces % 2)), ' ') << " | ";
    }
}

void headerWrite() {
    array<string, 5> border{}, headers = {"ID", "Brand", "Age", "Date",
" Status "};

    for (size_t i = 0; i < 5; ++i) {
        border[i] = string(spaceCount[i], '=');
    }

    cout << "| ";
    writeAndCenter(border);
    cout << "\n| ";
    writeAndCenter(headers);
    cout << "\n| ";
    writeAndCenter(border);
    cout << endl;
}

void list() { // Display list in console
    for (const auto& pair : inventory) {
        cout << "| ";
         // Extract stock data from second param of inventory (aka the
stockData class)
        array<string, 5> temp = pair.second.getAll();
        writeAndCenter(temp);
        cout << " " << endl;
    }
}

void listAll() { // Command: /list all
    headerWrite();
```

```
    list();
}

void listFilterWrite(array<string, 5>& temp) { // Command: /list filter
- sub1
    cout << "| ";
    writeAndCenter(temp);
    cout << " " << endl;
}

void listFilterType(const invMap& index, const string& filterValue) { //
Command: /list filter - sub2
    auto range = index.equal_range(filterValue);
    for (auto i = range.first; i != range.second; ++i) {
        array<string, 5> temp = i->second->getAll();
        listFilterWrite(temp);
    }
}


void listFilter(const string &filterType, const string &filterValue) {
//Comand: list filter
    headerWrite();
    static unordered_map<string, const invMap&> filterIndices = {
        {"brand", brandIndex},
        {"age", ageIndex},
        {"date", dateIndex},
        {"status", statusIndex}
    };

    auto category = filterIndices.find(filterType);
    if (category != filterIndices.end()) {
        listFilterType(category->second, filterValue);
    } else {
        cout << "Invalid filter type." << endl;
    }
}


bool isValidStockNumber(const string &stockNumber) {
    // Based on the csv, min numbers = 0 & min letters = 4, all length =
10
    return (stockNumber.length()== 10) && (find_if(stockNumber.begin(),
stockNumber.end(), ::isalpha) != stockNumber.end());
}
```

```
int main () {
    extractDataCSV();
    listFilter("status", "Sold"); // Test
    system("pause");
    return 0;
}
```

--- Working Iteration 3: Added some helper functions to use later on, functions to check if stockData params are valid, added storage for overwrite (not yet used) and discards, added error handling

```
#include <iostream>
#include <unordered_map>
#include <algorithm>
#include <fstream> // CSV Extract
#include <sstream>
#include <cstring>
#include <string>
#include <vector>
#include <array>

using namespace std;

// _____________________________________ HELPER FUNCTIONS
_____________________________________

// (not used)
bool isNum (const string &s){ // Check if int/longint
      // Check if string can be converted to long int
    try {
        stol(s);
    }
    catch (...) {
        return 0;
    }
      // Return T/F whether string still matches with long int
counterpart
    return (s == to_string(stol(s)));
}

bool caseInsensitive (const string &str1, const string &str2) { //
Compare 2 str (w/o case)
      // Return false early if length differs
      if (str1.size() == str2.size()) return 0;

      // Check each char if match regardless of
      for (size_t i = 0; i < str1.size(); i++) {
            if (tolower(str1[i]) == tolower(str2[i])) return 1;
      }

    return 0;
}
```

```
// ____________________________________ INVENTORY
____________________________________
class stockData {
    string id, brand, age, date, status;
    public:
        stockData() : id(""), brand(""), age(""), date(""), status("")
{} // Forgot to add a default constructor
        stockData (string &id_, string &brand_, string &age_, string
&date_, string &status_)
        : id(id_), brand(brand_), age(age_), date(date_),
status(status_){}

        // Force returned value to be read-only
        string getID() const { return id; }
        string getBrand() const { return brand; }
        string getAge() const { return age; }
        string getDate() const { return date; }
        string getStatus() const { return status; }
        array<string, 5> getAll() const {return {id, brand, age, date,
status}; }

};

unordered_map<string, stockData> inventory; // Key: ID -> stockData
vector<stockData> discards, overwritten; // Vector used since it's just
a small amount of Invalid Combinations
typedef unordered_multimap<string, stockData*> invMap;
invMap brandIndex, ageIndex, dateIndex, statusIndex; // Key: Category ->
stockData*

// ___________________________________ VALUE VALIDITY CHECKS
___________________________________

bool validID(const string &StockID) {
    // Based on the csv, min numbers = 0 & min letters = 4, all length =
10 | ∴ NEED len = 10, at least 1 letter
    return (StockID.length()== 10) && (find_if(StockID.begin(),
StockID.end(), ::isalpha) != StockID.end());
}

bool validDate(const string &StockDate) {
    // Check if extVal[0] (Date) has a length of 8, 9, or 10  (x/x/xxxx,
(xx/x) or x/xx/xxxx, xx/xx/xxxx)
    return (StockDate.length() >= 8) || (StockDate.length() <= 10);
}
```

```
bool validAge(const string &StockAge) {
    // Check if New / Old
    return (caseInsensitive(StockAge,"old")) ||
(caseInsensitive(StockAge,"new"));
}

bool validStatus(const string &StockStatus) {
    // Check if New / Old
    return (caseInsensitive(StockStatus,"on-hand")) ||
(caseInsensitive(StockStatus,"sold"));
}

// _____________________________ INVENTORY ERRORS
_____________________________

enum ErrorIndex {
    IN,       // Something is wrong with this data line during input
    IN_ID,    // Invalid Input ID
    IN_DATE, // Invalid Input Date
    IN_AGE,  // Invalid Input Age
    IN_STATUS, // Invalid Input Status
    FILE_LOAD, // Failed File Load
    FILTER_NOTFOUND, // Invalid Filter List Type
    UNKNOWN  // Unknown error type
};

// Define error messages as a constexpr array using const char*
constexpr array<const char*, 7> errorMessages = {{
    "Something is wrong with this data line, code added to discard
pile.",
    "Invalid ID detected at line ",
    "Invalid Date detected at line ",
    "Invalid Age detected at line ",
    "Invalid Status detected at line ",
    "Data Extraction Failed: CSV not opened.",
    "Invalid Filter Type. Filter Type does not match scope (Brand, Age,
Date, Status)."
}};

// Function to display error messages
void errInv(ErrorIndex errInd, int errline = 0) {
    if (errInd >= IN && errInd < UNKNOWN) {
        cerr << errorMessages[errInd];
        if (errInd != IN) {
            cerr << errline; // Append line number if applicable
        }
```

```
    } else {
        cerr << "Unknown error type.";
    }

    cerr << endl;
}

// _____________________________________ CSV EXTRACTS
_____________________________________

void extractDataCSV() {
    ifstream motorPHData("Copy of MotorPH Inventory Data - March 2023
Inventory Data.csv"); // simplified

    if (!motorPHData.is_open()) { // Check if csv data is accessible
        errInv(FILE_LOAD);
        exit(1);
    }

    string line{};
    array<string, 5> extVal{};
    static int currentLine = 2;

    getline(motorPHData, line);
    getline(motorPHData, line);

    while (getline(motorPHData, line)) {
        stringstream ss(line);

        ++currentLine;

        for (size_t i = 0; i < extVal.size(); ++i) {
            if (!getline(ss, extVal[i], ',')) break;
            // Per slot of extVal: runs getline(ss, extVal[i], ',')
            // ss until ',' is assigned to extVal[i]
            // if items be only 4, getline cant assign sstream to
extVal[4] and returns false bool (this is news to me)
            // false return triggers break
        }

        const string& stockID = extVal[3],
        stockDate = extVal[0],
        stockAge = extVal[1],
        stockStatus = extVal[4];

        bool invalidParam{};
```

```
        if (!validID(stockID)) {errInv(IN_ID, currentLine); invalidParam
= 1;};
        if (!validDate(stockDate)) {errInv(IN_DATE, currentLine);
invalidParam = 1;};
        if (!validAge(stockAge)) {errInv(IN_AGE, currentLine);
invalidParam = 1;};
        if (!validStatus(stockStatus)) {errInv(IN_STATUS, currentLine);
invalidParam = 1;};

        stockData newStock(extVal[3], extVal[2], extVal[1], extVal[0],
extVal[4]); // Use constructor

        if (invalidParam) {errInv(IN); discards.push_back(newStock);};

        // Indexing for fast filtering
        stockData* stockPtr = &inventory[stockID];
        brandIndex.emplace(stockPtr->getBrand(), stockPtr);
        ageIndex.emplace(stockPtr->getAge(), stockPtr);
        dateIndex.emplace(stockPtr->getDate(), stockPtr);
        statusIndex.emplace(stockPtr->getStatus(), stockPtr);
    }
}

// _____________________________________ INVENTORY COMMANDS
_____________________________________

array<int,5> spaceCount = {10, 8, 3, 10, 10}; // Number of spaces to
center

void writeAndCenter (array<string, 5> &temp) { // Write and Center
    for (size_t i = 0; i < 5; i++) {
        string currentString = temp[i];
        int spaces = (spaceCount[i]-currentString.length());
        cout << string(size_t(spaces/2), ' ') << currentString
            << string(size_t((spaces/2) + (spaces % 2)), ' ') << " | ";
    }
}

void headerWrite() { // Display Headers
    static array<string, 5> border{}, headers = {"ID", "Brand", "Age",
"Date", " Status "};

    for (size_t i = 0; i < 5; ++i) {
        border[i] = string(spaceCount[i], '=');
    }
```

```
    cout << "| ";
    writeAndCenter(border);
    cout << "\n| ";
    writeAndCenter(headers);
    cout << "\n| ";
    writeAndCenter(border);
    cout << endl;
}

void list() { // Display list in console
    for (const auto& pair : inventory) {
        cout << "| ";
         // Extract stock data from second param of inventory (aka the
stockData class)
        array<string, 5> temp = pair.second.getAll();
        writeAndCenter(temp);
        cout << " " << endl;
    }
}

void listAll() { // Command: /list all
    headerWrite();
    list();
}

void listFilterWrite(array<string, 5>& temp) { // Command: /list filter
- sub1
    cout << "| ";
    writeAndCenter(temp);
    cout << " " << endl;
}

void listFilterType(const invMap& index, const string& filterValue) { //
Command: /list filter - sub2
    auto range = index.equal_range(filterValue);
    for (auto i = range.first; i != range.second; ++i) {
        array<string, 5> temp = i->second->getAll();
        listFilterWrite(temp);
    }
}


void listFilter(const string &filterType, const string &filterValue) {
//Comand: list filter
    headerWrite();
    static unordered_map<string, const invMap&> filterIndices = {
```

```
        {"brand", brandIndex},
        {"age", ageIndex},
        {"date", dateIndex},
        {"status", statusIndex}
    };

    auto category = filterIndices.find(filterType);
    if (category != filterIndices.end()) { // If category returns a
invMap from valid key @ filterIndices
        listFilterType(category->second, filterValue); // send map and
value to listFilterType
    }
}

int main () {
    extractDataCSV();
    listFilter("status", "Sold");
    system("pause");
    return 0;
}
```

--- Working Iteration 4: Fixed caseInsensitive condition mistake, added deques for overwrites and discards, initialised invmaps (were previously not ini), added overwrite error handling, changed csv extraction (made inventory extraction parallel work under parallel threads, discards and overwritten data (which uses a different data structure) are extracted with the same function), added save/export csv, fixed variables that were previously shadowed, added timer to extraction function, discard and overwrite handling in inventory extraction, adjusted display functions, made filterValue case insensitive

```cpp
#include <iostream>
#include <unordered_map> // For inventory storage, key lookup and return
#include <algorithm> // to use find_if
#include <fstream> // CSV Extract and Saving
#include <sstream> // stringstream to get (near) O(1) display
#include <thread>
#include <string>
#include <vector>
#include <atomic>
#include <array>
#include <deque> // for discard piles
#include <mutex> // To avoid data racing

using namespace std;

// _____________________________________ HELPER FUNCTIONS
_____________________________________

// (not used)
bool isNum (const string &s){ // Check if int/longint
      // Check if string can be converted to long int
    try {
        stol(s);
    }
    catch (...) {
        return 0;
    }
      // Return T/F whether string still matches with long int
counterpart
    return (s == to_string(stol(s)));
}

string caseLower (string s) {
    transform(s.begin(), s.end(), s.begin(), ::tolower);
    return s;
}

bool caseInsensitive (const string &str1, const string &str2) { //
```

```
Compare 2 str (w/o case)
      // Return false early if length differs
      if (str1.size() != str2.size()) return 0;

      // Check each char if match regardless of
      for (size_t i = 0; i < str1.size(); i++) {
            if (tolower(str1[i]) == tolower(str2[i])) return 1;
      }

    return 0;
}

// ____________________________________ INVENTORY STORAGE
____________________________________

class stockData {
    string id, brand, age, date, status;
    public:
        stockData() : id(""), brand(""), age(""), date(""), status("")
{} // Forgot to add a default constructor
        stockData (string &id_, string &brand_, string &age_, string
&date_, string &status_)
        : id(id_), brand(brand_), age(age_), date(date_),
status(status_){}

        // Force returned value to be read-only
        string getID() const { return id; }
        string getBrand() const { return brand; }
        string getAge() const { return age; }
        string getDate() const { return date; }
        string getStatus() const { return status; }
        array<string, 5> getAll() const {return {id, brand, age, date,
status}; }

};

struct discardEntry {
    stockData data;
    string timestamp;
};

unordered_map<string, stockData> inventory{}; // Key: ID -> stockData
deque<discardEntry> discards{}, overwriteHistory{};
// Changed to deque since use case is good for stacks but still need the
index-calling ability of vectors
```

```
typedef unordered_multimap<string, stockData*> invMap;
invMap brandIndex{}, ageIndex{}, dateIndex{}, statusIndex{}; // Key:
Category -> stockData*


// _____________________________ VALUE VALIDITY CHECKS
_____________________________

bool validID(const string &StockID) {
    // Based on the csv, min numbers = 0 & min letters = 4, all length =
10 | ∴ NEED len = 10, at least 1 letter
    return (StockID.length()== 10) && (find_if(StockID.begin(),
StockID.end(), ::isalpha) != StockID.end());
}

bool validDate(const string &StockDate) {
    // Check if extVal[0] (Date) has a length of 8, 9, or 10  (x/x/xxxx,
(xx/x) or x/xx/xxxx, xx/xx/xxxx)
    return (StockDate.length() >= 8) || (StockDate.length() <= 10);
}

bool validAge(const string &StockAge) {
    // Check if New / Old
    return (caseInsensitive(StockAge,"old")) ||
(caseInsensitive(StockAge,"new"));
}

bool validStatus(const string &StockStatus) {
    // Check if New / Old
    return (caseInsensitive(StockStatus,"on-hand")) ||
(caseInsensitive(StockStatus,"sold"));
}

// _____________________________ INVENTORY ERRORS
_____________________________

mutex errorHandling;

enum ErrorIndex {
    IN,      // Something is wrong with this data line during input
    IN_ID,   // Invalid Input ID
    IN_DATE, // Invalid Input Date
    IN_AGE,  // Invalid Input Age
    IN_STATUS, // Invalid Input Status
    FILE_LOAD, // Failed File Load
    FILTER_NOTFOUND, // Invalid Filter List Type
```

```
    OVERWRITE, // For overwrite instances
    UNKNOWN  // Unknown error type
};

// Define error messages as a constexpr array using const char*
constexpr array<const char*, 8> errorMessages = {
    "Something is wrong with this data line, code added to discard
pile.",
    "Invalid ID detected at line ",
    "Invalid Date detected at line ",
    "Invalid Age detected at line ",
    "Invalid Status detected at line ",
    "Data Extraction Failed: CSV not opened. Filename: ",
    "Invalid Filter Type. Filter Type does not match scope (Brand, Age,
Date, Status).",
    "ID already exists. Overwriting data assigned to ID: "
};

// Function to display error messages
void errInv(ErrorIndex errInd, const string& errline = "") {
    lock_guard<mutex> coutLock(errorHandling);
    if (errInd >= IN && errInd < UNKNOWN) {
        cerr << errorMessages[errInd] << (!errline.empty() ? errline :
""); // Append line number if applicable
    } else {
        cerr << "Unknown error type.";
    }

    cerr << endl;
}

// ____________________________________ CSV EXTRACTS
____________________________________

mutex inventoryMutex, clockElapsed, storageAssign;

void extractDataCSV(
    const string& filename,
    unordered_map<string, stockData>* targetMap = nullptr,
    deque<discardEntry>* targetDeque = nullptr,
    int countHeaderSkip = 0,
    int currentLine = 1,
    int until = 0) {

    ifstream file(filename);
    if (!file.is_open()) {errInv(FILE_LOAD,filename); return;} // Stop
```

```
extraction and notify on failure to open file
    string line{}, timestamp{};
    array<string, 5> extVal{};

    auto start = Clock::now(); // Monitor the time when extraction
starts

    // Skip header lines if needed
    for (int i = 0; i < countHeaderSkip; ++i) getline(file, line);

    while (getline(file, line)) {
        //cout << whatThread << endl;
        if ((until != 0)&&(until == currentLine)) break;
        stringstream ss(line);

        for (size_t i = 0; i < extVal.size(); ++i) {
            if (!getline(ss, extVal[i], ',')) break;
            // Per slot of extVal: runs getline(ss, extVal[i], ',')
            // ss until ',' is assigned to extVal[i]
            // if items be only 4, getline cant assign sstream to
extVal[4] and returns false bool (this is news to me)
            // false return triggers break
        }
        string &id_ = extVal[3], &brand_ = extVal[2], &age_ = extVal[1],
&date_ = extVal[0], &status_ = extVal[4];

        stockData newData(id_, brand_, age_, date_, status_);
        // If current file have timestamps then just extract, else take
current time
        timestamp = (getline(ss, timestamp, ',') && (timestamp != "")) ?
timestamp :
to_string(chrono::system_clock::to_time_t(chrono::system_clock::now()));

        lock_guard<mutex> lock(storageAssign);
        bool invalidParam{};
        if (!validID(id_)) {errInv(IN_ID, to_string(currentLine));
invalidParam = 1;};
        if (!validDate(date_)) {errInv(IN_DATE, to_string(currentLine));
invalidParam = 1;};
        if (!validAge(age_)) {errInv(IN_AGE, to_string(currentLine));
invalidParam = 1;};
        if (!validStatus(status_)) {errInv(IN_STATUS,
to_string(currentLine)); invalidParam = 1;};

        stockData newStock(extVal[3], extVal[2], extVal[1], extVal[0],
extVal[4]); // Use constructor
```

```
        if (discards.size() > 15) discards.pop_back();
        if (invalidParam) {errInv(IN); discards.push_front({newStock,
timestamp}); continue;;};

        ++currentLine;

        if (targetMap) {
            lock_guard<mutex> lock(inventoryMutex); // added to avoid
data racing
            // If key already exists, add current to overwriteHistory
then overwrite
            if (targetMap->find(id_) != targetMap->end()) {
                overwriteHistory.push_front({(*targetMap)[id_],
timestamp});
                if (overwriteHistory.size() > 15)
overwriteHistory.pop_back(); // disregard older overwrites
                errInv(OVERWRITE, id_);
            }

            (*targetMap)[id_] = newData;

            // Indexing for fast filtering
            stockData* stockPtr = &((*targetMap)[id_]);
            brandIndex.insert({caseLower(brand_), stockPtr});
            ageIndex.insert({caseLower(age_), stockPtr});
            dateIndex.insert({caseLower(date_), stockPtr});
            statusIndex.insert({caseLower(status_), stockPtr});

            lock_guard<mutex> coutLock(clockElapsed);
        } else if (targetDeque) {
            targetDeque->push_front({newData, timestamp});
            if (targetDeque->size() > 15) targetDeque->pop_back();
        }
    }
    file.close();

    lock_guard<mutex> coutLock(clockElapsed);
    auto end = Clock::now();
    chrono::duration<double,milli> elapsed = end - start; // Tally time
taken and display
    cout << "Data extraction from " << filename << " completed in " <<
elapsed.count() << " ms.\n";
}

void parallelExtractInventory(const string& filename) {
```

```
    // Open file once to designate midpoint
    ifstream file(filename);
    if (!file.is_open()) {errInv(FILE_LOAD,filename); return;} // Stop
extraction and notify on failure to open file
    int lineCount{};
    string line;
    while (getline(file, line)) lineCount++;
    file.close();

    int midPoint = lineCount / 2, next = midPoint + 1;
    thread t1(extractDataCSV, filename, &inventory, nullptr, 2, 1,
midPoint);
    thread t2(extractDataCSV, filename, &inventory, nullptr, next, next,
0);

    t1.join();
    t2.join();
}

void mainExtraction() { // Just to compile extraction function calls
into one function call
    thread overwriteThread(extractDataCSV,
string("overwriteHistory.csv"), nullptr, &overwriteHistory, 0, 1, 0);
    thread discardThread(extractDataCSV, string("discards.csv"),
nullptr, &discards, 0, 1, 0);
    discardThread.join();
    overwriteThread.join();

    thread inventoryThread(parallelExtractInventory,
string("inventory.csv"));
    inventoryThread.join();
}

void saveDataCSV(const string& filename, const unordered_map<string,
stockData>& dataMap) {
    ofstream file(filename);
    for (const auto& pair : dataMap) {
        file << pair.second.getDate() << "," << pair.second.getAge() <<
","
             << pair.second.getBrand() << "," << pair.second.getID() <<
","
             << pair.second.getStatus() << "\n";
    }
    file.close();
}
```

```
void saveDataCSV(const string& filename, const deque<discardEntry>&
dataDeque) {
    ofstream file(filename);
    for (const auto& entry : dataDeque) {
        file << entry.data.getDate() << "," << entry.data.getAge() <<
","
             << entry.data.getBrand() << "," << entry.data.getID() <<
","
             << entry.data.getStatus() << "," << entry.timestamp <<
"\n";
    }
    file.close();
}
// _____________________________________ DISPLAY
_____________________________________

const static array<int, 5> spaceCount = {10, 8, 3, 10, 10};
static stringstream dispInv;

void writeAndCenter(array<string, 5> temp) {
    dispInv << " | ";
    for (size_t i = 0; i < 5; i++) {
        string currentString = temp[i];
        size_t spaces = (currentString.length() >= spaceCount[i]) ? 0 :
spaceCount[i] - currentString.length();
        dispInv << string(spaces / 2, ' ') << currentString
                << string((spaces / 2) + (spaces % 2), ' ') << " | ";
    }
    dispInv << endl;
}

string hBorders(const int& charNum) {
    return string(charNum, '=');
}

const static array<string, 5> horizontalBorders = {
    hBorders(spaceCount[0]),
    hBorders(spaceCount[1]),
    hBorders(spaceCount[2]),
    hBorders(spaceCount[3]),
    hBorders(spaceCount[4])
};

const static array<string, 5> headers = {"ID", "Brand", "Age", "Date",
"Status"};
```

```
void headerWrite() {
    writeAndCenter(horizontalBorders);
    writeAndCenter(headers);
    writeAndCenter(horizontalBorders);
}


void listAll() {
    for (const auto& pair : inventory) {
        array<string, 5> item = pair.second.getAll();
        writeAndCenter(item);
    }
}

void listFilterType(const invMap& index, const string& filterValue) { //
Command: /list filter - sub2
    auto range = index.equal_range(caseLower(filterValue));
    for (auto i = range.first; i != range.second; ++i) {
        array<string, 5> temp = i->second->getAll();
        writeAndCenter(temp);
    }
}

void listFilter(const string& filterType, const string& filterValue) {
//Comand: list filter
    static unordered_map<string, const invMap&> filterIndices = {
        {"brand", brandIndex},
        {"age", ageIndex},
        {"date", dateIndex},
        {"status", statusIndex}
    };

    auto category = filterIndices.find(filterType);
    if (category != filterIndices.end()) { // If category returns a
invMap from valid key @ filterIndices
        cout << "Found valid key" << endl;
        listFilterType(category->second, filterValue); // send map and
value to listFilterType
    }
}

enum listType {LIST_ALL, LIST_FILTER};

void displayInventory(listType listCmd, string param1 = "", string
param2 = "") {
```

```
    dispInv.str(""); // Clear the stringstream

    headerWrite();
    switch (listCmd) {
        case 0:
            listAll();
            break;
        case 1:
            listFilter(param1,param2);
            break;
    }

    cout << dispInv.str() << endl;
}
// ____________________________________ INVENTORY COMMANDS
_____________________________________

int main () {
    mainExtraction();
    // listFilter("status", "Sold");
    system("pause");
    displayInventory(LIST_ALL);
    system("pause");
    displayInventory(LIST_FILTER, "brand","Honda");
    system("pause");
    displayInventory(LIST_FILTER, "brand","honda");
    system("pause");

    saveDataCSV("inventory.csv", inventory);
    saveDataCSV("discards.csv", discards);
    saveDataCSV("overwriteHistory.csv", overwriteHistory);
    return 0;
}
```

--- Working Iteration 5: command handling, stockdata setter, discardentry constructor, error handling for invalid command, changed const char*s to string_view, added caseUp() for uppercase conversion, simplified caseInsensitive(), added splitCommand() for command tokenisation, added extractDateKey() for date processing, added forEach macro for range loops, added SortingData struct for sorting, added ParsedCSVLine struct for CSV parsing, fixed validDate() logic,

Split CSV processing into: parseCSVLine() (lines to stockData elements), validateFields() (check if elements are valid), extractDataCSV() (main extraction function)

added radix sort, added merge sort, added map for index return for presorted brand list, replaced single dispInv with multiple cached streams, added mutexes for each display cache (still not sure how mutexes work— (aka this might've not been necessary), added combineDisplayCaches() utility, implemented parallel cache updating with threads, command processing, added trigUpdate() for cache refresh, added overwrite history tracking, implemented proper case handling for IDs, added timestamping for discards/overwrites, fixed CSV parsing edge cases, container size checks, preload header format, added main menu, added proper program exit

```
#include <iostream> // cout, cin, cerr
#include <unordered_map> // For inventory storage, key lookup and return
#include <algorithm> // to use transform, find_if, sort
#include <iomanip> // setw (space setter)
#include <fstream> // CSV Extract and Saving (ifstream, ofstream)
#include <sstream> // stringstream to get (near) O(1) display
#include <thread> // multithreading for csv extraction and cache
processing
#include <string>
#include <vector>
#include <array>
#include <deque> // for discard piles
#include <mutex> // To avoid data racing

using namespace std;
using Clock = chrono::high_resolution_clock;

// _____________________________ HELPER FUNCTIONS
_____________________________

// (not used)
bool isNum (const string &s){ // Check if int/longint
      // Check if string can be converted to long int
    try {
        stol(s);
    }
    catch (...) {
```

```
        return 0;
    }
      // Return T/F whether string still matches with long int
counterpart
    return (s == to_string(stol(s)));
}

string caseLower (string s) {
    transform(s.begin(), s.end(), s.begin(), ::tolower);
    return s;
}

string caseUp (string s) {
    transform(s.begin(), s.end(), s.begin(), ::toupper);
    return s;
}

bool caseInsensitive (const string &str1, const string &str2) { //
Compare 2 str (w/o case)
      // Return false early if length differs then check each char if
match regardless of
      return (str1.size() != str2.size()) ? 0 : (caseLower(str1) !=
caseLower(str2)) ? 0 : 1;
}

vector<string> splitCommand(const string& s) {
    vector<string> tokens;
    string token;
    istringstream iss(s);
    while (iss >> token) {
        tokens.push_back(token);
    }
    return tokens;
}

int extractDateKey(const string& dateStr) {
    stringstream ss(dateStr);
    string part;
    vector<int> parts;
    while (getline(ss, part, '/')) {
        parts.push_back(stoi(part));
    }
    if (parts.size() != 3) return 0;
    return parts[2] * 10000 + parts[1] * 100 + parts[0];
}
```

```
void trigUpdate();

#define forEach(item, in) for (const auto& item : in)
// ____________________________________ INVENTORY STORAGE
____________________________________

class stockData {
    string id, brand, age, date, status;
    public:
        stockData() : id(""), brand(""), age(""), date(""), status("")
{} // Forgot to add a default constructor
        stockData (string &id_, string &brand_, string &age_, string
&date_, string &status_)
        : id(id_), brand(brand_), age(age_), date(date_),
status(status_){}

        // Force returned value to be read-only
        string getID() const { return id; }
        string getBrand() const { return brand; }
        string getAge() const { return age; }
        string getDate() const { return date; }
        string getStatus() const { return status; }
        array<string, 5> getAll() const {return {id, brand, age, date,
status}; }

        // Setters
        void setID(const string &val) {id = val;}
        void setBrand(const string &val) {brand = val;}
        void setAge(const string &val) {age = val;}
        void setDate(const string &val) {date = val;}
        void setStatus(const string &val) {status = val;}

};

struct discardEntry {
    stockData data;
    string timestamp;
    discardEntry(const stockData& d, const string& t)
        : data(d), timestamp(t) {}
};

unordered_map<string, stockData> inventory{}; // Key: ID -> stockData
deque<discardEntry> discards{}, overwriteHistory{};
// Changed to deque since use case is good for stacks but still need the
index-calling ability of vectors
```

```
typedef unordered_multimap<string, stockData*> invMap;
invMap brandIndex{}, ageIndex{}, dateIndex{}, statusIndex{}; // Key:
Category -> stockData*


// _____________________________ VALUE VALIDITY CHECKS
_____________________________

bool validID(const string &StockID) {
    // Based on the csv, min numbers = 0 & min letters = 4, all length =
10 | ∴ NEED len = 10, at least 1 letter
    return (StockID.length()== 10) && (find_if(StockID.begin(),
StockID.end(), ::isalpha) != StockID.end());
}

bool validDate(const string &StockDate) {
    // Check if extVal[0] (Date) has a length of 8, 9, or 10  (x/x/xxxx,
(xx/x) or x/xx/xxxx, xx/xx/xxxx)
    return (StockDate.length() >= 8) && (StockDate.length() <= 10);
}

bool validAge(const string &StockAge) {
    // Check if New / Old
    return (caseInsensitive(StockAge,"old")) ||
(caseInsensitive(StockAge,"new"));
}

bool validStatus(const string &StockStatus) {
    // Check if New / Old
    return (caseInsensitive(StockStatus,"on-hand")) ||
(caseInsensitive(StockStatus,"sold"));
}

// _____________________________ INVENTORY ERRORS
_____________________________

mutex errorHandling;

enum ErrorIndex {
    IN,       // Something is wrong with this data line during input
    IN_ID,    // Invalid Input ID
    IN_DATE, // Invalid Input Date
    IN_AGE,  // Invalid Input Age
    IN_STATUS, // Invalid Input Status
    FILE_LOAD, // Failed File Load
    FILTER_NOTFOUND, // Invalid Filter List Type
```

```
    OVERWRITE, // For overwrite instances
    INVALID_COMMAND,
    UNKNOWN  // Unknown error type
};

// Define error messages as a constexpr array using string_view

constexpr array<string_view, 9> errorMessages = {
    "Something is wrong with this data line, code added to discard
pile.",
    "Invalid ID detected at line ",
    "Invalid Date detected at line ",
    "Invalid Age detected at line ",
    "Invalid Status detected at line ",
    "Data Extraction Failed: CSV not opened. Filename: ",
    "Invalid Filter Type. Filter Type does not match scope (Brand, Age,
Date, Status).",
    "ID already exists. Overwriting data assigned to ID: ",
    "Use '/help Inventory' to list valid commands. This command is
either unknown or invalid: "
};

// Function to display error messages
void errInv(ErrorIndex errInd, const string& errline = "") {
    lock_guard<mutex> coutLock(errorHandling);
    if (errInd >= IN && errInd < UNKNOWN) {
        cerr << errorMessages[errInd] << (!errline.empty() ? errline :
""); // Append line number if applicable
    } else {
        cerr << "Unknown error type.";
    }

    cerr << endl;
}

// ____________________________________ CSV EXTRACTS
____________________________________

mutex inventoryMutex, clockElapsed, storageAssign;

// CSV Line Parsing

struct ParsedCSVLine {
    array<string, 5> fields{}; // Expected order: date, age, brand, id,
status.
    string timestamp{}; // Timestamp field - if empty, we'll fill it
```

```
with the current time.
};

ParsedCSVLine parseCSVLine(const string& line) {
    ParsedCSVLine result;
    stringstream ss(line);

    // Extract 5 main fields
    for(auto& field : result.fields)
        getline(ss, field, ',');

    // Extract optional timestamp or use current
    getline(ss, result.timestamp, ',');
    if(result.timestamp.empty())
        result.timestamp =
to_string(chrono::system_clock::to_time_t(chrono::system_clock::now()));

    return result;
}

bool validateFields(const ParsedCSVLine& parsed, int lineNum) {
    bool valid = true;
    if(!validID(parsed.fields[3]))    { errInv(IN_ID,
to_string(lineNum)); valid = false; }
    if(!validDate(parsed.fields[0])) { errInv(IN_DATE,
to_string(lineNum)); valid = false; }
    if(!validAge(parsed.fields[1]))  { errInv(IN_AGE,
to_string(lineNum)); valid = false; }
    if(!validStatus(parsed.fields[4])){ errInv(IN_STATUS,
to_string(lineNum)); valid = false; }
    return valid;
}

template<typename Container>
void handleValidData(const ParsedCSVLine& parsed, Container* container,
const stockData& data) {
    if constexpr(is_same_v<Container, unordered_map<string, stockData>>)
{
        lock_guard<mutex> mapLock(inventoryMutex);
        if(auto it = container->find(data.getID()); it !=
container->end()) {
            overwriteHistory.push_front({it->second, parsed.timestamp});
            if(overwriteHistory.size() > 15)
overwriteHistory.pop_back();
            errInv(OVERWRITE, data.getID());
        }
```

```
        auto [iter, _] = container->insert_or_assign(data.getID(),
data);

        // Update indexes
        stockData* ptr = &iter->second;
        brandIndex.insert({caseLower(ptr->getBrand()), ptr});
        ageIndex.insert({caseLower(ptr->getAge()), ptr});
        dateIndex.insert({caseLower(ptr->getDate()), ptr});
        statusIndex.insert({caseLower(ptr->getStatus()), ptr});
    }
    else if constexpr(is_same_v<Container, deque<discardEntry>>) {
        container->push_front({data, parsed.timestamp});
        if(container->size() > 15) container->pop_back();
    }
}

template<typename Container>
void extractDataCSV(const string& fileName, Container* target, int
skipHeaders = 0, int startLine = 1, int until = 0) {
    ifstream file(fileName
);
    if(!file.is_open()) { errInv(FILE_LOAD, fileName
); return; }

    string line;
    for(int i = 0; i < skipHeaders; ++i) getline(file, line);

    auto tStart = Clock::now();
    size_t currentLine = startLine;

    while(getline(file, line) && (until == 0 || currentLine < until)) {
        ParsedCSVLine parsed = parseCSVLine(line);
        stockData data(parsed.fields[3], parsed.fields[2],
                       parsed.fields[1], parsed.fields[0],
parsed.fields[4]);

        {
            lock_guard<mutex> globalLock(storageAssign);

            if(!validateFields(parsed, currentLine)) {
                discards.emplace_front(data, parsed.timestamp);
                if(discards.size() > 15) discards.pop_back();
                continue;
            }

            if(target) handleValidData(parsed, target, data);
```

```
            currentLine++;
        }
    }

    lock_guard<mutex> coutLock(clockElapsed);
    auto duration = chrono::duration<double, milli>(
        Clock::now() - tStart
    ).count();
    cout << "Data extraction from " << fileName
 << " completed in "
         << duration << " ms.\n";
}


void parallelExtractInventory(const string& fileName
) {

    // Open file once to designate midpoint
    ifstream file(fileName
);
    if (!file.is_open()) {errInv(FILE_LOAD,fileName
); return;} // Stop extraction and notify on failure to open file
    int lineCount{};
    string line;
    while (getline(file, line)) lineCount++;
    file.close();

    int midPoint = lineCount / 2, next = midPoint + 1;
    thread t1(&extractDataCSV<unordered_map<string, stockData>>,
fileName
, &inventory, 2, 1, midPoint);
    thread t2(&extractDataCSV<unordered_map<string, stockData>>,
fileName
, &inventory, next, next, 0);

    t1.join();
    t2.join();
}

void mainExtraction() { // Just to compile extraction function calls
into one function call
    thread overwriteThread(&extractDataCSV<deque<discardEntry>>,
string("overwriteHistory.csv"), &overwriteHistory, 0, 1, 0);
    thread discardThread(&extractDataCSV<deque<discardEntry>>,
string("discards.csv"), &discards, 0, 1, 0);
    discardThread.join();
```

```
    overwriteThread.join();

    thread inventoryThread(parallelExtractInventory,
string("inventory.csv"));
    inventoryThread.join();
    trigUpdate();
}

void saveDataCSV(const string& fileName
, const unordered_map<string, stockData>& dataMap) {
    ofstream file(fileName
);
    file << "\n";
    forEach(pair, dataMap) {
        file  << "\n" << pair.second.getDate() << "," <<
pair.second.getAge() << ","
            << pair.second.getBrand() << "," << pair.second.getID() <<
","
            << pair.second.getStatus();
    }
    file.close();
}

void saveDataCSV(const string& fileName
, const deque<discardEntry>& dataDeque) {
    ofstream file(fileName
);
    forEach(entry, dataDeque) {
        file << entry.data.getDate() << "," << entry.data.getAge() <<
","
            << entry.data.getBrand() << "," << entry.data.getID() <<
","
            << entry.data.getStatus() << "," << entry.timestamp <<
"\n";
    }
    file.close();
}

// ____________________________________ SORTING
____________________________________
struct SortingData {
    const stockData* data; // Pointer to the original stockData
    int dateKey;           // Date in YYYYMMDD format
    int brandKey;          // Alphabetical index of the brand
    string idKey;          // Original ID for alphanumeric sorting
};
```

```
vector<SortingData> sortedInventory; // Global vector to hold sorted
data
unordered_map<string, int> brandMap; // Maps brand names to their
alphabetical index
mutex brandMapMutex, sortedInventoryMutex;

void buildBrandMap() {
    lock_guard<mutex> lock(brandMapMutex);
    vector<string> brands;
    forEach(item, inventory) { // for each item in inv: check if brand
is already in vector, append if not found
        string brand = item.second.getBrand();
        if (find(brands.begin(), brands.end(), brand) == brands.end()) {
            brands.push_back(brand);
        }
    }
    sort(brands.begin(), brands.end()); // C++ introsort --> (it's so
easy, why don't we just use this for all of it???)
    brandMap.clear(); // reset brand map before assigning
    for (size_t i = 0; i < brands.size(); ++i) {
        brandMap[brands[i]] = i;
    }
}

void prepareSortingData() {
    lock_guard<mutex> lock(sortedInventoryMutex);
    sortedInventory.clear();
    forEach(item, inventory) {
        const stockData& data = item.second;
        sortedInventory.push_back({
            &data,
            extractDateKey(data.getDate()),
            brandMap[data.getBrand()],
            data.getID()
        });
    }
}

enum SortKey { DATE_KEY, BRAND_KEY };

void radixSort(vector<SortingData>& data, SortKey key) {
    const int BASE = 10;
    int maxVal{};

    forEach(item, data) { // Get highest key
```

```
        int currentKey;
        switch (key) {
            case DATE_KEY: currentKey = item.dateKey; break;
            case BRAND_KEY: currentKey = item.brandKey; break;
            default: currentKey = 0;
        }
        if (currentKey > maxVal) maxVal = currentKey;
    }

    for (int exp = 1; maxVal / exp > 0; exp *= BASE) {
        vector<SortingData> output(data.size());
        vector<int> count(BASE, 0);

        forEach(item, data) {
            int currentKey;
            switch (key) {
                case DATE_KEY: currentKey = item.dateKey; break;
                case BRAND_KEY: currentKey = item.brandKey; break;
                default: currentKey = 0;
            }
            int digit = (currentKey / exp) % BASE;
            count[digit]++;
        }

        for (int i = 1; i < BASE; ++i) count[i] += count[i-1];
        for (int i = data.size()-1; i >=0; --i) {
            int currentKey;
            switch (key) {
                case DATE_KEY: currentKey = data[i].dateKey; break;
                case BRAND_KEY: currentKey = data[i].brandKey; break;
                default: currentKey = 0;
            }
            int digit = (currentKey / exp) % BASE;
            output[--count[digit]] = data[i];
        }

        data = output;
    }
}

void mergeAlphanumeric(vector<SortingData>& data, int left, int mid, int
right) {
    vector<SortingData> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right) {
```

```
        if (data[i].idKey < data[j].idKey) {
            temp[k++] = data[i++];
        } else {
            temp[k++] = data[j++];
        }
    }

    while (i <= mid) temp[k++] = data[i++];
    while (j <= right) temp[k++] = data[j++];

    for (k = 0; k < temp.size(); k++) {
        data[left + k] = temp[k];
    }
}

void mergeSortAlphanumeric(vector<SortingData>& data, int left, int
right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSortAlphanumeric(data, left, mid);
        mergeSortAlphanumeric(data, mid + 1, right);
        mergeAlphanumeric(data, left, mid, right);
    }
}

// ____________________________________ DISPLAY
____________________________________

static constexpr array<int, 5> spaceCount = {10, 8, 3, 10, 10};

static constexpr string_view headerBlock =
    " | ========== | ======== | === | ========== | ========== |\n"
    " |     ID     |  Brand   | Age |    Date    |   Status   |\n"
    " | ========== | ======== | === | ========== | ========== |\n";

static constexpr string_view bottomBorder =
    " | ========== | ======== | === | ========== | ========== |\n";

static constexpr string_view headerBlock_DiscardPile =
    " | ======= | ========== | ======== | === | ========== | ==========
| ========== |\n"
    " |  Index  |     ID     |  Brand   | Age |    Date    |   Status
|  Timestamp  |\n"
    " | ======= | ========== | ======== | === | ========== | ==========
| ========== |\n";
```

```
static constexpr string_view bottomBorder_DiscardPile =
    " | ========== | ======== | === | ========== | ========== |\n";


static constexpr string_view invHelpDisp =
"\n=============================================================================
====================================="
"\n/help Inventory \t#shows slash commands below:"
"\n/add [stock number] [brand] [age] [status] [date] \t# Add new stock"
"\n/update age [stock number] [age] \t# Updates available stock's age "
"\n/update brand [stock number] [brand] \t# Updates ... brand"
"\n/update date [stock number] [date] \t# Updates ... date"
"\n/update status [stock number] [status] \t# Updates ... status"
"\n/remove [stock number] \t# Removes stock and adds it to discard pile"
"\n/restore list [discard / overwrite] \t# Displays discard/overwrite
pile"
"\n/restore [discard / overwrite] [index] \t# Adds stock from
discard/overwrite pile to inventory"
"\n/list all \t# Displays all stocks"
"\n/list filter [brand / age / status / date] [val to filter] \t#
Displays all stocks under that filter"
"\n/sort [id / brand / age / status / date] \t# Displays all stocks
(sorted by specification)"
"\n/xsearch [stock number] \t# Exact search for id and displays info
regarding that id"
"\n/fsearch [stock number] \t# Fuzzy search ..."
"\n/mm \t# Return to main menu"
"\n/exit \t# Safely exits the program"
"\n=============================================================================
=====================================";

template<typename T, size_t N>
void writeAndCenter(stringstream& ss, const array<T,N>& temp) {
    ss << " | ";
    for (size_t i = 0; i < 5; i++) {
        string currentString = temp[i];
        size_t spaces = (currentString.length() >= 10) ? 0 : 10 -
currentString.length();
        ss << string(spaces / 2, ' ') << currentString << string((spaces
/ 2) + (spaces % 2), ' ') << " | ";
    }
    ss << endl;
}

void listFilterType(const invMap& index, const string& filterValue,
stringstream& ss) { // Command: /list filter - sub
```

```
    auto range = index.equal_range(caseLower(filterValue));
    for (auto i = range.first; i != range.second; ++i) {
        array<string, 5> temp = i->second->getAll();
        writeAndCenter(ss, temp);
    }
}

void listFilter(const string& filterType, const string& filterValue,
stringstream& ss) { //Comand: list filter
    static unordered_map<string, const invMap&> filterIndices = {
        {"brand", brandIndex}, {"age", ageIndex}, {"date", dateIndex},
{"status", statusIndex}
    };

    ss.str("");
    ss << headerBlock;

    auto category = filterIndices.find(filterType);
    if (category != filterIndices.end()) { // If category returns a
invMap from valid key @ filterIndices
        listFilterType(category->second, filterValue, ss);// send map
and value to listFilterType
    }

    ss << bottomBorder;
}

void displaySortedInventory(stringstream& ss, const vector<SortingData>&
sortedData) {
    ss << headerBlock;
    for (const auto& item : sortedData) {
        array<string, 5> data = item.data->getAll();
        writeAndCenter(ss, data);
    }
    ss << bottomBorder;
}
// ____________________________________ DISPLAY CACHE
____________________________________

stringstream dispList, dispList_Age1, dispList_Age2, dispList_Status1,
dispList_Status2,
    dispSort_ID, dispSort_Brand, dispSort_Age, dispSort_Date,
dispSort_Status;

mutex dispListMutex, dispList_Age1Mutex, dispList_Age2Mutex,
dispList_Status1Mutex, dispList_Status2Mutex,
```

```
    dispSort_IDMutex, dispSort_BrandMutex, dispSort_AgeMutex,
dispSort_DateMutex, dispSort_StatusMutex;

void appendEntries(const string& content, stringstream& output) {
    istringstream iss(content);
    string line;
    vector<string> lines;

    while (getline(iss, line)) {lines.push_back(line);}

    // Verify that the content has sufficient lines (minimum of four:
header, at least one entry, and footer).
    if (lines.size() >= 4) {
        for (size_t i = 3; i < lines.size() - 1; ++i) {output <<
lines[i] << '\n';}
    }
}

void combineDisplayCaches(stringstream& src1, mutex& src1Mutex,
                          stringstream& src2, mutex& src2Mutex,
                          stringstream& dest, mutex& destMutex) {
    // Lock both source mutexes to ensure safe read operations.
    lock_guard<mutex> lock1(src1Mutex);
    lock_guard<mutex> lock2(src2Mutex);

    stringstream combined;
    combined << headerBlock; // prepend the header block.

    appendEntries(src1.str(), combined);
    appendEntries(src2.str(), combined);

    combined << bottomBorder; // Append the bottom border to conclude
the output.

    // Lock the destination mutex before updating the destination
stream.
    lock_guard<mutex> lockDest(destMutex);
    dest.str(combined.str());
}

void trigUpdate() {
    auto tStart = Clock::now();
    buildBrandMap();
    prepareSortingData();

    vector<thread> threads;
```

```
    // ListAll
    threads.emplace_back([&]() {
            vector<SortingData> local = sortedInventory;
            sort(local.begin(), local.end(), [](const SortingData& a,
const SortingData& b) {
                if (a.dateKey != b.dateKey) {
                    return a.dateKey < b.dateKey;
                } else {
                    return a.idKey < b.idKey;
                }
            });
            stringstream ss;
            displaySortedInventory(ss, local);
            lock_guard<mutex> lock(dispListMutex);
            dispList.str(ss.str());
        });

    // Corrected addUpdThread macro definition
    #define addUpdThread(filterType, filterValue, mx, stream) \
        threads.emplace_back([&]() { \
            stringstream ss; \
            listFilter(filterType, filterValue, ss); \
            lock_guard<mutex> lock(mx); \
            stream.str(ss.str()); \
        })

    // Age filters
    addUpdThread("age", "new", dispList_Age1Mutex, dispList_Age1);
    addUpdThread("age", "old", dispList_Age2Mutex, dispList_Age2);

    // Status filter
    addUpdThread("status", "on-hand", dispList_Status1Mutex,
dispList_Status1);
    addUpdThread("status", "sold", dispList_Status2Mutex,
dispList_Status2);

    // Combine age filters into dispSort_Age
    threads.emplace_back([&]() {
    combineDisplayCaches(dispList_Age1, dispList_Age1Mutex,
                         dispList_Age2, dispList_Age2Mutex,
                         dispSort_Age, dispSort_AgeMutex);
        });

    // Combine status filters into dispSort_Status
    threads.emplace_back([&]() {
```

```
    combineDisplayCaches(dispList_Status1, dispList_Status1Mutex,
                         dispList_Status2, dispList_Status2Mutex,
                         dispSort_Status, dispSort_StatusMutex);
        });

    // Sorted lists
    threads.emplace_back([&]() {
        vector<SortingData> local = sortedInventory;
        mergeSortAlphanumeric(local, 0, local.size()-1);
        stringstream ss;
        displaySortedInventory(ss, local);
        lock_guard<mutex> lock(dispSort_IDMutex);
        dispSort_ID.str(ss.str());
    });

    threads.emplace_back([&]() { // Sort by Brand
        vector<SortingData> local = sortedInventory;
        radixSort(local, BRAND_KEY);
        stringstream ss;
        displaySortedInventory(ss, local);
        lock_guard<mutex> lock(dispSort_BrandMutex);
        dispSort_Brand.str(ss.str());
    });

    threads.emplace_back([&]() { // Sort by Date
        vector<SortingData> local = sortedInventory;
        radixSort(local, DATE_KEY);
        stringstream ss;
        displaySortedInventory(ss, local);
        lock_guard<mutex> lock(dispSort_DateMutex);
        dispSort_Date.str(ss.str());
    });

    for (auto& t : threads) t.join();
    lock_guard<mutex> coutLock(clockElapsed);
    auto duration = chrono::duration<double, milli>(Clock::now() -
tStart).count();
    cout << "Cache load completed in "
         << duration << " ms.\n";
}
// ____________________________________ INVENTORY COMMANDS
____________________________________

#define grabTokens const vector<string> &tokens

void quit() {
```

```
    saveDataCSV("inventory.csv", inventory);
    saveDataCSV("discards.csv", discards);
    saveDataCSV("overwriteHistory.csv", overwriteHistory);
    exit(0);
}

void commandInvHelp(grabTokens) {
    if (tokens.size() == 1) cout << "Command usage: /help
[Inventory/inv/Payroll/pr]\n";
    else if ((tokens[1] == "inventory") || (tokens[1] == "inv")) cout <<
invHelpDisp << endl;
    else if ((tokens[1] == "payroll") || (tokens[1] == "pr")) cout <<
"Payroll part of the program isn't finished yet" << endl;
    else errInv(INVALID_COMMAND, "Command usage: /help
[Inventory/inv/Payroll/pr]");
}

void commandAdd(grabTokens) {
    if (tokens.size() != 6) {
        errInv(INVALID_COMMAND, "Usage: /add [stock number] [brand]
[age] [status] [date]");
        return;
    }

    string id = caseUp(tokens[1]), brand = tokens[2], age = tokens[3],
status = tokens[4], date = tokens[5];

    if (!validID(id)) { errInv(IN_ID, "Invalid ID: " + id); return; }
    if (!validDate(date)) { errInv(IN_DATE, "Invalid date: " + date);
return; }
    if (!validAge(age)) { errInv(IN_AGE, "Invalid age: " + age); return;
}
    if (!validStatus(status)) { errInv(IN_STATUS, "Invalid status: " +
status); return; }

    lock_guard<mutex> lock(inventoryMutex);
    auto existing = inventory.find(id);
    if (existing != inventory.end()) {
        stockData& oldData = existing->second;
        // Remove old entries from indexes
        auto removeFromIndex = [&oldData](auto& index, const string&
key) {
            auto range = index.equal_range(key);
            for (auto it = range.first; it != range.second; ) {
                if (it->second == &oldData) it = index.erase(it);
                else ++it;
```

```
            }
        };
        removeFromIndex(brandIndex, caseLower(oldData.getBrand()));
        removeFromIndex(ageIndex, caseLower(oldData.getAge()));
        removeFromIndex(dateIndex, caseLower(oldData.getDate()));
        removeFromIndex(statusIndex, caseLower(oldData.getStatus()));
        // Add to overwriteHistory
        overwriteHistory.emplace_front(oldData,
to_string(chrono::system_clock::to_time_t(chrono::system_clock::now())))
;
        if (overwriteHistory.size() > 15) overwriteHistory.pop_back();
        inventory.erase(existing);
    }

    stockData newData(id, brand, age, date, status);
    inventory[id] = newData;
    stockData* ptr = &inventory[id];
    brandIndex.emplace(caseLower(brand), ptr);
    ageIndex.emplace(caseLower(age), ptr);
    dateIndex.emplace(caseLower(date), ptr);
    statusIndex.emplace(caseLower(status), ptr);
    trigUpdate();
}

void commandUpdate(grabTokens) {
    if (tokens.size() < 4) { // Ensure there are enough tokens
        errInv(INVALID_COMMAND, "Usage: /update [age/brand/date/status]
[stock number] [value]");
        return;
    }
    string field = tokens[1], id = caseUp(tokens[2]), newVal =
tokens[3];
    newVal[0] = toupper(newVal[0]);
    if (!validID(id)) { errInv(IN_ID, "Invalid ID: " + id); return; }

    lock_guard<mutex> lock(inventoryMutex);
    auto it = inventory.find(id);
    if (it == inventory.end()) {
        errInv(INVALID_COMMAND, "Stock ID not found: " + id);
        return;
    }

    stockData& data = it->second;
    string oldBrand = data.getBrand();
    string oldAge = data.getAge();
    string oldDate = data.getDate();
```

```
    string oldStatus = data.getStatus();

    if (field == "age") {
        if (!validAge(newVal)) {
            errInv(IN_AGE, "Invalid age: " + newVal);
            return;
        }
        data.setAge(newVal);
    } else if (field == "brand") {
        data.setBrand(newVal);
    } else if (field == "date") {
        if (!validDate(newVal)) {
            errInv(IN_DATE, "Invalid date: " + newVal);
            return;
        }
        data.setDate(newVal);
    } else if (field == "status") {
        if (!validStatus(newVal)) {
            errInv(IN_STATUS, "Invalid status: " + newVal);
            return;
        }
        data.setStatus(newVal);
    } else {
        errInv(INVALID_COMMAND, "Invalid update field: " + field);
        return;
    }

    overwriteHistory.emplace_front(data,
to_string(chrono::system_clock::to_time_t(chrono::system_clock::now())))
;
    if (overwriteHistory.size() > 15) overwriteHistory.pop_back();

    // Remove old entries from indexes
    auto removeFromIndex = [&](auto& index, const string& key) {
        auto range = index.equal_range(key);
        for (auto it = range.first; it != range.second; ) {
            if (it->second == &data) {
                it = index.erase(it);
            } else {
                ++it;
            }
        }
    };

    removeFromIndex(brandIndex, caseLower(oldBrand));
    removeFromIndex(ageIndex, caseLower(oldAge));
```

```
    removeFromIndex(dateIndex, caseLower(oldDate));
    removeFromIndex(statusIndex, caseLower(oldStatus));

    // Insert new entries
    brandIndex.emplace(caseLower(data.getBrand()), &data);
    ageIndex.emplace(caseLower(data.getAge()), &data);
    dateIndex.emplace(caseLower(data.getDate()), &data);
    statusIndex.emplace(caseLower(data.getStatus()), &data);

    trigUpdate();
}

void commandRemove(grabTokens) {
    if (tokens.size() != 2) {
        errInv(INVALID_COMMAND, "Usage: /remove [stock number]");
        return;
    }

    string id = caseUp(tokens[1]);
    if (!validID(id)) { errInv(IN_ID, "Invalid ID: " + id); return; }

    lock_guard<mutex> lock(inventoryMutex);
    auto it = inventory.find(id);
    if (it == inventory.end()) {
        errInv(INVALID_COMMAND, "Stock ID not found: " + id);
        return;
    }

    discards.emplace_front(it->second,
to_string(chrono::system_clock::to_time_t(chrono::system_clock::now())))
;
    if (discards.size() > 15) discards.pop_back();

    // Remove from indexes
    auto removeFromIndex = [&](auto& index, const string& key) {
        auto range = index.equal_range(key);
        for (auto i = range.first; i != range.second; ) {
            if (i->second == &it->second) i = index.erase(i);
            else ++i;
        }
    };
    removeFromIndex(brandIndex, caseLower(it->second.getBrand()));
    removeFromIndex(ageIndex, caseLower(it->second.getAge()));
    removeFromIndex(dateIndex, caseLower(it->second.getDate()));
    removeFromIndex(statusIndex, caseLower(it->second.getStatus()));
```

```
    inventory.erase(it);
    trigUpdate();
}

void commandRestore(grabTokens) {
    if (tokens.size() < 2) {
        errInv(INVALID_COMMAND, "Usage: /restore list
[discard/overwrite] or /restore [discard/overwrite] [index]");
        return;
    }

    if (tokens[1] == "list") {
        if (tokens.size() != 3) {
            errInv(INVALID_COMMAND, "Usage: /restore list
[discard/overwrite]");
            return;
        }

        const deque<discardEntry>* pile = (tokens[2] == "discard") ?
&discards : &overwriteHistory;
        stringstream ss;
        ss << headerBlock_DiscardPile;
        for (size_t i = 0; i < pile->size(); ++i) {
            const auto& entry = (*pile)[i];
            ss << " | " << setw(6) << i << " | " << setw(10) <<
entry.data.getID() << " | "
               << setw(8) << entry.data.getBrand() << " | " << setw(3)
<< entry.data.getAge() << " | "
               << setw(10) << entry.data.getDate() << " | " << setw(10)
<< entry.data.getStatus()
               << " | " << setw(11) << entry.timestamp << " |\n";
        }
        ss << bottomBorder_DiscardPile;
        cout << ss.str();
    } else {
        if (tokens.size() != 3) {
            errInv(INVALID_COMMAND, "Usage: /restore [discard/overwrite]
[index]");
            return;
        }

        int index;
        try { index = stoi(tokens[2]); }
        catch (...) { errInv(INVALID_COMMAND, "Invalid index: " +
tokens[2]); return; }
```

```
        deque<discardEntry>* pile = (tokens[1] == "discard") ? &discards
: &overwriteHistory;
        if (index < 0 || index >= pile->size()) {
            errInv(INVALID_COMMAND, "Index out of range: " +
to_string(index));
            return;
        }

        auto entry = (*pile)[index];
        lock_guard<mutex> lock(inventoryMutex);
        auto existing = inventory.find(entry.data.getID());
        if (existing != inventory.end()) {
            overwriteHistory.push_front(discardEntry(existing->second,
to_string(chrono::system_clock::now().time_since_epoch().count())));
            if (overwriteHistory.size() > 15)
overwriteHistory.pop_back();
            inventory.erase(existing);
        }

        inventory[entry.data.getID()] = entry.data;
        stockData* ptr = &inventory[entry.data.getID()];
        brandIndex.emplace(caseLower(ptr->getBrand()), ptr);
        ageIndex.emplace(caseLower(ptr->getAge()), ptr);
        dateIndex.emplace(caseLower(ptr->getDate()), ptr);
        statusIndex.emplace(caseLower(ptr->getStatus()), ptr);
        pile->erase(pile->begin() + index);
        trigUpdate();
    }
}

void commandList(grabTokens) {
    if (tokens.size() < 2) {
        errInv(INVALID_COMMAND, "Usage: /list [all/filter]");
        return;
    }

    if (tokens[1] == "all") {
        lock_guard<mutex> lock(dispListMutex);
        cout << dispList.str();
    } else if (tokens[1] == "filter" && tokens.size() == 4) {
        string filterType = tokens[2], filterValue = tokens[3];
        stringstream ss;
        if (filterType == "brand") listFilterType(brandIndex,
filterValue, ss);
        else if (filterType == "age") listFilterType(ageIndex,
filterValue, ss);
```

```
        else if (filterType == "status") listFilterType(statusIndex,
filterValue, ss);
        else if (filterType == "date") listFilterType(dateIndex,
filterValue, ss);
        else { errInv(FILTER_NOTFOUND, "Invalid filter type: " +
filterType); return; }
        cout << ss.str();
    } else {
        errInv(INVALID_COMMAND, "Invalid subcommand for /list");
    }
}

void commandSort(grabTokens) {
    if (tokens.size() != 2) {
        errInv(INVALID_COMMAND, "Usage: /sort
[id/brand/age/status/date]");
        return;
    }

    string key = tokens[1];
    if (key == "id") { lock_guard<mutex> lock(dispSort_IDMutex); cout <<
dispSort_ID.str(); }
    else if (key == "brand") { lock_guard<mutex>
lock(dispSort_BrandMutex); cout << dispSort_Brand.str(); }
    else if (key == "age") { lock_guard<mutex> lock(dispSort_AgeMutex);
cout << dispSort_Age.str(); }
    else if (key == "status") { lock_guard<mutex>
lock(dispSort_StatusMutex); cout << dispSort_Status.str(); }
    else if (key == "date") { lock_guard<mutex>
lock(dispSort_DateMutex); cout << dispSort_Date.str(); }
    else errInv(INVALID_COMMAND, "Invalid sort key: " + key);
}

void commandSearch(grabTokens) {
    if (tokens.size() != 2) {
        errInv(INVALID_COMMAND, "Usage: /xsearch [stock number]");
        return;
    }

    string id = caseUp(tokens[1]);
    if (!validID(id)) { errInv(IN_ID, "Invalid ID: " + id); return; }

    lock_guard<mutex> lock(inventoryMutex);
    auto it = inventory.find(id);
    if (it == inventory.end()) {
        cout << "Stock ID not found: " << id << endl;
```

```
        return;
    }

    array<string, 5> data = it->second.getAll();
    stringstream ss;
    ss << headerBlock;
    writeAndCenter(ss, data);
    ss << bottomBorder;
    cout << ss.str();
}

void processInventoryCommands() {
    string command;
    cout << "Please input a command to continue (use '/help Inventory'
for assistance).\n";


    while (true) {
        cout << "> ";
        getline(cin, command);
        vector<string> tokens = splitCommand(caseLower(command));
            if (tokens.empty()) continue; // Request for input again


        // The amount of elif here is killing me ;-;
        if (tokens[0] == "/help") commandInvHelp(tokens);
        else if (tokens[0] == "/add") commandAdd(tokens);
        else if (tokens[0] == "/update") commandUpdate(tokens);
        else if (tokens[0] == "/remove") commandRemove(tokens);
        else if (tokens[0] == "/restore") commandRestore(tokens);
        else if (tokens[0] == "/list") commandList(tokens);
        else if (tokens[0] == "/sort") commandSort(tokens);
        else if (tokens[0] == "/xsearch") commandSearch(tokens);
        else if (tokens[0] == "/fsearch") commandSearch(tokens);
        else if (tokens[0] == "/mm") return;
        else if (tokens[0] == "/exit") quit();
        else {errInv(INVALID_COMMAND, command);}
    }
}

void mainMenu() {
    string input;
    while (true) {
        cout << "Main menu:\n[1] Inventory\n[2] Payroll\n[3] Exit\n";
        cout << "Enter choice: ";
        getline(cin, input);
```

```
        input = caseLower(input);

        if (input == "1" || input == "i" || input == "inv" || input ==
"inventory") {
            mainExtraction();
            processInventoryCommands();
        } else if (input == "2" || input == "p" || input == "pr" ||
input == "payroll") {
            cout << "This part of the program is not yet done.\n";
        } else if (input == "3" || input == "exit") {
            quit();
        } else {
            cout << "Invalid choice. Please try again.\n";
        }
    }
}

// ____________________________________________ MAIN
_____________________________________________

int main() {
    // listFilter("status", "Sold");
    system("pause");
    mainMenu();
    return 0;
}
```

Working Iteration 6:

Changes:
Made header dynamic again (due to Brand not having a max length)
Fixed Restore List Header Misalignment
Pause Clear on New Command Prompt
Fixed Missing Borders for List All
Date Sorting Mishap Fixed
Added caseCap Helper Function
Fixed Add and Update Casing
Changed Command Handling From Elif to Map
Fixed Crashing on /add, /update, /restore, /remove Commands
Fixed Search Function
Implemented Fuzzy Search using Wagner-Fischer to get Levenshtein Distance
Preservation of Line 1 and 2 on CSV Save
Updated Date Validity
Invalid Line File Specification
Minimised Code Redundancy
Fixed multiple macros
Fuzzy ID for /update and /remove
Command History
Made some function inline
Added some aliases
Fixed Timestamp Processing
(+ a lot of bug squishing & optimisations I forgot to note)

```cpp
#include <iostream> // cout, cin, cerr
#include <unordered_map> // For inventory storage, key lookup and return
#include <functional> // for map elif alternative
#include <algorithm> // to use transform, find_if, sort (reluctantly), min
#include <iomanip> // setw (space setter)
#include <fstream> // CSV Extract and Saving (ifstream, ofstream)
#include <sstream> // stringstream to get (near) O(1) display
#include <numeric> // iota
#include <thread> // multithreading for csv extraction and cache processing
#include <string>
#include <vector>
#include <stack>
#include <ctime> // For time_t and localtime
#include <array>
#include <deque> // for discard piles
#include <mutex> // To avoid data racing
#include <set>

using namespace std;
```

```
using Clock = chrono::high_resolution_clock;
// ____________________________________ HELPER FUNCTIONS
____________________________________

inline string caseLower (string s) {
    if (!s.empty()) transform(s.begin(), s.end(), s.begin(), ::tolower);
    return s;
}

inline string caseUp (string s) {
    if (!s.empty()) transform(s.begin(), s.end(), s.begin(), ::toupper);
    return s;
}

inline string caseCap (string s) {
    if (!s.empty()) s[0] = static_cast<char>(toupper(s[0]));
    return s;
}

constexpr bool caseInsensitive(string_view str1, string_view str2) noexcept {
    return str1.size() == str2.size() &&
        equal(str1.begin(), str1.end(), str2.begin(),
            [](char a, char b) { return tolower(a) == tolower(b); });
}

constexpr bool isNumeric(string_view s) noexcept {
    return !s.empty() && all_of(s.begin(), s.end(), ::isdigit);
}

vector<string> splitCommand(const string& s) {
    vector<string> tokenCollection;
    tokenCollection.reserve(7);
    string token;
    istringstream iss(s);
    while (iss >> token) {
        tokenCollection.push_back(token);
    }
    return tokenCollection;
}

int extractDateKey(const string& dateStr) {
    stringstream ss(dateStr);
```

```
    string part;
    vector<int> dateParts;
    while (getline(ss, part, '/')) {
        dateParts.push_back(stoi(part));
    }
    if (dateParts.size() != 3) return 0;
    return dateParts[2] * 10000 + dateParts[0] * 100 + dateParts[1];
}

int levDist(const string &source, const string &target, int threshold =
INT_MAX) {
    //get lengths of both strings
    const size_t sourceLength = source.size(), targetLength = target.size();

    // If either strings are empty, return the length of the other
    // (basically if s = 0 all insertions elif t = 0 all deletions)
    if (sourceLength == 0) return static_cast<int>(targetLength);
    if (targetLength == 0) return static_cast<int>(sourceLength);

    // Create a 1D DP array for storing edit distances for the current row (TC
cant be optimised so SC instead)
    // you don't really need the full matrix, just the prev to compute the
next lines
    // https://www.youtube.com/watch?v=d-Eq6x1yssU
    vector<int> editDistanceRow(targetLength + 1);
    iota(editDistanceRow.begin(), editDistanceRow.end(), 0); // ini base case:
converting an empty string to prefixes of target

    // Process each character of the source string
    for (size_t srcIndex = 1; srcIndex <= sourceLength; ++srcIndex) {
        int rowMin = INT_MAX;
        int previousDiagonal = editDistanceRow[0]; // Store previous diagonal
for substitution calculation
        editDistanceRow[0] = static_cast<int>(srcIndex); // The first element
in each row represents deletions

        // Process each character of the target string
        for (size_t tgtIndex = 1; tgtIndex <= targetLength; ++tgtIndex) {
            int temp = editDistanceRow[tgtIndex]; // Store current value
before updating
```

```
            // Determine substitution cost (0 if characters match, 1 if they
differ)
            int substitutionCost = (source[srcIndex - 1] == target[tgtIndex -
1]) ? 0 : 1;

            // Compute the minimum of three operations:
            // 1. insert (editDistanceRow[tgtIndex - 1] + 1)
            // 2. delete (editDistanceRow[tgtIndex] + 1)
            // 3. substitute (previousDiagonal + substitutionCost)
            editDistanceRow[tgtIndex] = min({
                editDistanceRow[tgtIndex] + 1,      // del cost
                editDistanceRow[tgtIndex - 1] + 1,  // ins cost
                previousDiagonal + substitutionCost // sub cost
            });

            // Update previous diagonal for the next iteration
            previousDiagonal = temp;

            // Update rowMin with the minimum value in the current row
            rowMin = min(rowMin, editDistanceRow[tgtIndex]);
        }

        if (rowMin > threshold) return threshold + 1;
    }
    return editDistanceRow[targetLength];
}

inline bool confirmAction(const string& action, const string& id) {
    string input;
    cout << "Are you sure you want to " << action << " the stock with ID: " <<
id << "? (yes/no): ";
    while (getline(cin, input)) {
        if (caseLower(input) == "yes" || caseLower(input) == "y") return true;
        if (caseLower(input) == "no" || caseLower(input) == "n") return false;
        cout << "Invalid input. Please enter 'yes' or 'no': " << endl;
    }
    return false;
}

void trigUpdate();

#define forEach(item, in) for (const auto& item : in)
```

```
#define currentTime
chrono::system_clock::to_time_t(chrono::system_clock::now())
#define onFind(varname, find, end, ...) \
do { \
    auto varname = find; \
    if (varname != end) { \
        __VA_ARGS__ \
    } \
} while (0)

// _____________________________________ INVENTORY STORAGE
_____________________________________

void addBrandToSet(const string& newBrandLower);

class stockData {
    string id, brand, age, date, status;
    string brandLower, ageLower, dateLower, statusLower; // Lowercase versions
public:
    stockData() : id(""), brand(""), age(""), date(""), status("") {} //
Default constructor
    stockData(const string &id_, const string &brand_, const string &age_,
const string &date_, const string &status_)
        : id(id_), brand(brand_), age(age_), date(date_), status(status_) {
        brandLower = caseLower(brand);
        ageLower = caseLower(age);
        dateLower = caseLower(date);
        statusLower = caseLower(status);
        addBrandToSet(brandLower); // Add brand to set when creating stockData
    }

    // Getters
    string getID() const { return id; }
    string getBrand() const { return brand; }
    string getAge() const { return age; }
    string getDate() const { return date; }
    string getStatus() const { return status; }
    string getBrandLower() const { return brandLower; }
    string getAgeLower() const { return ageLower; }
    string getDateLower() const { return dateLower; }
    string getStatusLower() const { return statusLower; }
    array<string, 5> getAll() const { return {id, brand, age, date, status}; }
```

```
    // Setters
    void setID(const string &val) { id = val; }
    void setBrand(const string &val) {
        brand = val;
        brandLower = caseLower(val);
        addBrandToSet(brandLower); // Add brand to set when setting brand
    }
    void setAge(const string &val) {
        age = val;
        ageLower = caseLower(val);
    }
    void setDate(const string &val) {
        date = val;
        dateLower = caseLower(val);
    }
    void setStatus(const string &val) {
        status = val;
        statusLower = caseLower(val);
    }
};

struct discardEntry {
    stockData data;
    time_t timestamp;
    discardEntry(const stockData& d, time_t t)
        : data(d), timestamp(t) {}
};

unordered_map<string, stockData> inventory{}; // Key: ID -> stockData
deque<discardEntry> discards{}, overwriteHistory{};
// Changed to deque since use case is good for stacks but still need the
index-calling ability of vectors

typedef unordered_multimap<string, stockData*> invMap;
invMap brandIndex{}, ageIndex{}, dateIndex{}, statusIndex{}; // Key: Category
-> stockData*

static size_t brandMaxLength = 10;

// _____________________________ VALUE VALIDITY CHECKS
_____________________________
```

```
bool validID(const string &StockID) {
    // Based on the csv, min numbers = 0 & min letters = 4, all length = 10 |
∴ NEED len = 10, at least 1 letter
    return (StockID.length()== 10) && (find_if(StockID.begin(), StockID.end(),
::isalpha) != StockID.end());
}

bool validDate(const string &StockDate) {
    // Check if extVal[0] (Date) has a length of 8, 9, or 10
    if ((StockDate.length() < 8) || (StockDate.length() > 10)) return false;

    // Check if data between / is a number
    size_t i{};
    string part;
    array<string, 3> date;
    stringstream ss(StockDate);
    while(i < 3 && getline(ss, part, '/')) {
        if (!isNumeric(part)) return false;
        date[i++] = part;
    }

    // Check exactly 3 parts and no leftover data
    if (i != 3 || getline(ss, part)) return false;

    // Set date values
    int year = stoi(date[2]);
    int month = stoi(date[0]);
    int day = stoi(date[1]);

    // Check if date is within range
    if ((year < 0) || (month > 12) || (month < 1) || (day < 1)) return false;

    // Leap year check
    auto checkLeapYr = [](int yr) {
        return (yr % 4 == 0 && yr % 100 != 0) || (yr % 400 == 0);
    };

    // Check if day is within month range
    int maxDay;
    switch (month) {
        case 2: maxDay = (checkLeapYr(year) ? 29 : 28); break;
```

```
        case 1:
        case 3:
        case 5:
        case 7:
        case 8:
        case 10:
        case 12: maxDay = 31; break;
        case 4:
        case 6:
        case 9:
        case 11: maxDay = 30; break;
        default: return false;
    }
    return (day <= maxDay);
}

bool validAge(const string &StockAge) {
    // Check if New / Old
    return (caseInsensitive(StockAge,"old")) ||
(caseInsensitive(StockAge,"new"));
}

bool validStatus(const string &StockStatus) {
    // Check if New / Old
    return (caseInsensitive(StockStatus,"on-hand")) ||
(caseInsensitive(StockStatus,"sold"));
}

// _____________________________________ INVENTORY ERRORS
_____________________________________

mutex inventoryMutex, errorHandling, coutMutex;

enum ErrorIndex {
    IN,       // Something is wrong with this data line during input
    IN_ID,    // Invalid Input ID
    IN_DATE, // Invalid Input Date
    IN_AGE,  // Invalid Input Age
    IN_STATUS, // Invalid Input Status
    FILE_LOAD, // Failed File Load
    FILTER_NOTFOUND, // Invalid Filter List Type
    OVERWRITE, // For overwrite instances
```

```
    INVALID_COMMAND,
    UNKNOWN  // Unknown error type
};

// Define error messages as a constexpr array using string_view

constexpr array<string_view, 9> errorMessages = {
    "Something is wrong with this data line, code added to discard pile.",
    "Invalid ID detected at line ",
    "Invalid Date detected at line ",
    "Invalid Age detected at line ",
    "Invalid Status detected at line ",
    "Data Extraction Failed: CSV not opened. Filename: ",
    "Invalid Filter Type. Filter Type does not match scope (Brand, Age, Date,
Status).",
    "ID already exists. Overwriting data assigned to ID: ",
    "Use '/help Inventory' to list valid commands. This command is either
unknown or invalid: "
};

// Function to display error messages
void errInv(ErrorIndex errInd, const string& errline = "") {
    lock_guard<mutex> lock(coutMutex);
    lock_guard<mutex> coutLock(errorHandling);
    if (errInd >= IN && errInd < UNKNOWN) {
        cerr << errorMessages[errInd] << (!errline.empty() ? errline : ""); //
Append line number if applicable
    } else {
        cerr << "Unknown error type.";
    }
    cerr << endl;
}

// ____________________________________ CSV EXTRACTS
____________________________________

mutex clockElapsed, storageAssign;
mutex brandIndexMutex, ageIndexMutex, dateIndexMutex, statusIndexMutex; // For
indexes

// CSV Line Parsing
```

```
struct ParsedCSVLine {
    array<string, 5> fields{}; // Expected order: date, age, brand, id,
status.
    string timestamp{}; // Timestamp field – if empty, we'll fill it with the
current time.
};

ParsedCSVLine parseCSVLine(const string& line) {
    ParsedCSVLine result;
    stringstream ss(line);

    // Extract 5 main fields
    for(auto& field : result.fields) if (!getline(ss, field, ',')) break;

    // Extract optional timestamp or use current
    getline(ss, result.timestamp, ',');
    size_t end = result.timestamp.find_last_not_of(" \t\n\r");
    if (end != string::npos) {
        result.timestamp = result.timestamp.substr(0, end + 1);
    } else {
        result.timestamp.clear();
    }

    if (result.timestamp.empty()) {
        result.timestamp = to_string(currentTime);
    }

    return result;
}

bool validateFields(const ParsedCSVLine& parsed, size_t &lineNum, const
string_view &fileName) {
    bool valid = true;
    auto response = [&valid, &lineNum, &fileName](const ErrorIndex &errorType)
{
        errInv(errorType, to_string(lineNum));
        cerr << "(Error found at file: " << fileName << ")" << endl;
        valid = false;
    };

    if(!validID(parsed.fields[3])) response(IN_ID);
    if(!validDate(parsed.fields[0])) response(IN_DATE);
```

```
    if(!validAge(parsed.fields[1])) response(IN_AGE);
    if(!validStatus(parsed.fields[4])) response(IN_STATUS);
    return valid;
}

template<typename Mutex, typename Index>
void addToIndex(Mutex& mtx, Index& idx, const string& key, stockData* ptr) {
    lock_guard<mutex> lock(mtx);
    idx.emplace(key, ptr);
}


void addToIndexes(stockData* ptr) {
    addToIndex(brandIndexMutex, brandIndex, ptr->getBrandLower(), ptr);
    addToIndex(ageIndexMutex, ageIndex, ptr->getAgeLower(), ptr);
    addToIndex(dateIndexMutex, dateIndex, ptr->getDateLower(), ptr);
    addToIndex(statusIndexMutex, statusIndex, ptr->getStatusLower(), ptr);
}


void removeFromIndexes(stockData& oldData, bool isCmdAdd = false) {
    // Lambda to remove old entries from an index
    auto removeFromIndex = [&oldData, isCmdAdd](auto& index, const string&
key, auto& indexMutex) {
        if (!isCmdAdd) lock_guard<mutex> lock(indexMutex);
        auto range = index.equal_range(key);
        for (auto iter = range.first; iter != range.second; ) {
            if (iter->second == &oldData) iter = index.erase(iter);
            else ++iter;
        }
    };

    // Remove old entries from all indexes
    removeFromIndex(brandIndex, oldData.getBrandLower(), brandIndexMutex);
    removeFromIndex(ageIndex, oldData.getAgeLower(), ageIndexMutex);
    removeFromIndex(dateIndex, oldData.getDateLower(), dateIndexMutex);
    removeFromIndex(statusIndex, oldData.getStatusLower(), statusIndexMutex);
}

// Function to convert string to time_t
time_t stringToTimeT(const string& timestampStr) {
    if (isNumeric(timestampStr)) {
```

```
        return static_cast<time_t>(stol(timestampStr));
    } else {
        tm tm = {};
        istringstream ss(timestampStr);
        ss >> get_time(&tm, "%Y-%m-%d %H:%M:%S");
        return mktime(&tm);
    }
}

// Modify the handleValidData function
template<typename Container>
void handleValidData(Container* container, const stockData& data) {
    lock_guard<mutex> invLock(inventoryMutex);
    if constexpr(is_same_v<Container, unordered_map<string, stockData>>) {
        if(auto it = container->find(data.getID()); it != container->end()) {
            // Extract old data and remove from indexes
            stockData& oldData = it->second;
            removeFromIndexes(oldData);

            // Add to overwrite history and erase from inventory
            overwriteHistory.emplace_front(oldData, currentTime);
            if (overwriteHistory.size() > 16) overwriteHistory.pop_back();
            errInv(OVERWRITE, data.getID());
            container->erase(it); // Remove old data from inventory
        }

        // Insert new data and update indexes
        auto [iter, _] = container->insert_or_assign(data.getID(), data);
        stockData* ptr = &iter->second;
        addToIndexes(ptr);
    }
    else if constexpr(is_same_v<Container, deque<discardEntry>>) {
        time_t timestamp = currentTime; // Ensure currentTime is used as
time_t
        container->emplace_front(data, timestamp);
        if(container->size() > 16) container->pop_back();
    }
}

static size_t maxEntries = 30;
mutex maxEntriesMutex;
```

```
template<typename Container>
void extractDataCSV(const string& fileName, Container* target, int skipHeaders
= 0, size_t startLine = 1, int until = 0, int threadInstance = 0) {
    ifstream file(fileName);
    if(!file.is_open()) { errInv(FILE_LOAD, fileName); return; }

    string line;
    for (size_t i = 0; i < static_cast<size_t>(skipHeaders); ++i)
getline(file, line);

    auto tStart = Clock::now();
    size_t currentLine = startLine + static_cast<size_t>(skipHeaders);

    vector<stockData> localValidEntries;
    {
        lock_guard<mutex> maxEntriesLock(maxEntriesMutex);
        if (maxEntries < static_cast<size_t>(until)) maxEntries =
static_cast<size_t>(until);
        localValidEntries.reserve(maxEntries);
    }
    deque<discardEntry> localDiscards;

    while (getline(file, line) && (until == 0 || currentLine <
static_cast<size_t>(until))) {
        if (localValidEntries.size() == localValidEntries.capacity())
localValidEntries.reserve(localValidEntries.capacity() * 2);
        ParsedCSVLine parsed = parseCSVLine(line);
        stockData data(parsed.fields[3], parsed.fields[2],
                       parsed.fields[1], parsed.fields[0], parsed.fields[4]);
        {
            lock_guard<mutex> globalLock(storageAssign);
            if(!validateFields(parsed, currentLine, fileName)) {
                time_t timestamp = stringToTimeT(parsed.timestamp);
                localDiscards.emplace_front(data, timestamp);
                if(localDiscards.size() > 16) localDiscards.pop_back();
            } else if (target) {
                if constexpr (is_same_v<Container, deque<discardEntry>>) {
                    time_t timestamp = stringToTimeT(parsed.timestamp);
                    localDiscards.emplace_front(data, timestamp);
                    if (localDiscards.size() > 16) localDiscards.pop_back();
                } else {
                    localValidEntries.push_back(data);
```

```
                }
            }
            currentLine++;
        }
    }
    {
        lock_guard<mutex> globalLock(storageAssign);
        for (auto& data : localValidEntries) handleValidData(target, data);
        for (auto& entry : localDiscards) { discards.push_front(entry);
if(discards.size() > 16) discards.pop_back(); }
    }

    lock_guard<mutex> coutLock(clockElapsed);
    auto duration = chrono::duration<double, milli>(Clock::now() -
tStart).count();
    lock_guard<mutex> lock(coutMutex);
    string threadName = (threadInstance == 0) ? "Main Thread" : "Thread " +
to_string(threadInstance);
    clog << "Data extraction from " << fileName  << " (" << threadName << ")"
         << " completed in " << duration << " ms.\n";
}

void parallelExtractInventory(const string& fileName) {
    // Open file once to designate midpoint
    ifstream file(fileName);
    if (!file.is_open()) {errInv(FILE_LOAD,fileName); return;} // Stop
extraction and notify on failure to open file

    int lineCount = 0;
    string line;
    while (getline(file, line)) lineCount++;
    file.close();

    const int headers = 2;
    int dataLines = lineCount - headers;
    if (dataLines <= 0) {
        clog << "No data lines to process." << endl;
        return;
    }

    int midPoint = dataLines / 2;
```

```
    // Thread 1 processes first half of data lines (after headers)
    int t1_skipHeaders = headers;
    size_t t1_startLine = 1;
    int t1_until = headers + midPoint;

    // Thread 2 processes second half
    int t2_skipHeaders = headers + midPoint;
    size_t t2_startLine = static_cast<size_t>(midPoint) + 1;
    int t2_until = 0; // Read until end

    thread t1(&extractDataCSV<unordered_map<string, stockData>>, fileName,
&inventory,
              t1_skipHeaders, t1_startLine, t1_until, 1);
    thread t2(&extractDataCSV<unordered_map<string, stockData>>, fileName,
&inventory,
              t2_skipHeaders, t2_startLine, t2_until, 2);

    if (t1.joinable()) t1.join();
    if (t2.joinable()) t2.join();
}

void mainExtraction() { // Just to compile extraction function calls into one
function call
    clog << "Extracting data from CSV files..." << endl;
    thread overwriteThread(&extractDataCSV<deque<discardEntry>>,
string("overwriteHistory.csv"), &overwriteHistory, 0, 1, 0, 0);
    thread discardThread(&extractDataCSV<deque<discardEntry>>,
string("discards.csv"), &discards, 0, 1, 0, 0);
    discardThread.join();
    overwriteThread.join();

    thread inventoryThread(parallelExtractInventory, string("inventory.csv"));
    inventoryThread.join();
    trigUpdate();
}

void saveDataCSV(const string& fileName, const unordered_map<string,
stockData>& dataMap) {
    ifstream inFile(fileName); // Open file to preserve headers
    vector<string> headers;
    string line;
```

```
    // Grab the first two lines
    for (int i = 0; i < 2 && getline(inFile, line); ++i) {
        headers.push_back(line);
    }
    inFile.close();

    ofstream outFile(fileName);
    // Write preserved headers
    forEach(fileLine, headers) {
        outFile << fileLine << '\n';
    }

    // Write data
    forEach(pair, dataMap) {
        outFile << pair.second.getDate() << "," << pair.second.getAge() << ","
                << pair.second.getBrand() << "," << pair.second.getID() << ","
                << pair.second.getStatus() << "\n";
    }
    outFile.close();
}

void saveDataCSV(const string& fileName, const deque<discardEntry>& dataDeque)
{
    ofstream file(fileName);
    forEach(entry, dataDeque) {
        time_t ts = entry.timestamp;
        tm* tm_info = localtime(&ts);
        char buffer[20];
        strftime(buffer, sizeof(buffer), "%Y-%m-%d %H:%M:%S", tm_info);
        file << entry.data.getDate() << "," << entry.data.getAge() << ","
             << entry.data.getBrand() << "," << entry.data.getID() << ","
             << entry.data.getStatus() << "," << entry.timestamp << "\n";
    }
    file.close();
}

// _____________________________________ SORTING
_____________________________________

struct SortingData {
    const stockData* data; // Pointer to the original stockData
    int dateKey;           // Date in YYYYMMDD format
```

```
    int brandKey;          // Alphabetical index of the brand
    int ageKey;            // 0 for New, 1 for Old
    int statusKey;         // 0 for On-Hand, 1 for Sold
    string idKey;          // Original ID for alphanumeric sorting
};

vector<SortingData> sortedInventory; // Global vector to hold sorted data
unordered_map<string, int> brandMap; // Maps brand names to their alphabetical
index
mutex brandMapMutex, sortedInventoryMutex;
static set<string> brandsSet; // Maintains sorted order

void buildBrandMap() {
    lock_guard<mutex> lock(brandMapMutex);
    brandMap.clear();
    int index = 0;
    forEach(brand, brandsSet) {
        brandMap[brand] = index++;
        if (brand.length() > brandMaxLength) brandMaxLength = brand.length();
    }
}

// When adding a new brand:
void addBrandToSet(const string& newBrandLower) {
    {
        lock_guard<mutex> lock(brandMapMutex);
        brandsSet.insert(newBrandLower);
    }
    buildBrandMap();
}

void prepareSortingData() {
    lock_guard<mutex> invLock(inventoryMutex);
    lock_guard<mutex> lock(sortedInventoryMutex);
    lock_guard<mutex> brandLock(brandMapMutex);
    sortedInventory.clear();
    sortedInventory.reserve(inventory.size());
    forEach(item, inventory) {
        const stockData& data = item.second;
        sortedInventory.push_back({
            &data,
            extractDateKey(data.getDate()),
```

```
            brandMap[data.getBrandLower()],
            (caseLower(data.getAge()) == "new" ? 0 : 1),        // ageKey: 0
for New, 1 for Old
            (caseLower(data.getStatus()) == "on-hand" ? 0 : 1), // statusKey:
0 for On-Hand, 1 for Sold
            data.getID()
        });
    }
}

enum SortKey { DATE_KEY, BRAND_KEY, AGE_KEY, STATUS_KEY , ID_KEY};

void radixSort(vector<SortingData>& data, SortKey key) {
    const int BASE = 10;
    int maxVal{};

    forEach(item, data) { // Get highest key
        int currentKey;
        switch (key) {
            case DATE_KEY: currentKey = item.dateKey; break;
            case BRAND_KEY: currentKey = item.brandKey; break;
            case AGE_KEY: currentKey = item.ageKey; break;
            case STATUS_KEY: currentKey = item.statusKey; break;
            default: currentKey = 0;
        }
        if (currentKey > maxVal) maxVal = currentKey;
    }

    for (int exp = 1; maxVal / exp > 0; exp *= BASE) {
        vector<SortingData> output(data.size());
        vector<size_t> count(BASE, 0);

        forEach(item, data) {
            int currentKey;
            switch (key) {
                case DATE_KEY: currentKey = item.dateKey; break;
                case BRAND_KEY: currentKey = item.brandKey; break;
                case AGE_KEY: currentKey = item.ageKey; break;
                case STATUS_KEY: currentKey = item.statusKey; break;
                default: currentKey = 0;
            }
            size_t digit = static_cast<size_t>((currentKey / exp) % BASE);
```

```
            count[digit]++;
        }

        for (size_t i = 1; i < static_cast<size_t>(BASE); ++i) count[i] +=
count[i-1];
        for (size_t idx = data.size(); idx-- > 0; ) {
            int currentKey;
            switch (key) {
                case DATE_KEY: currentKey = data[idx].dateKey; break;
                case BRAND_KEY: currentKey = data[idx].brandKey; break;
                case AGE_KEY: currentKey = data[idx].ageKey; break;
                case STATUS_KEY: currentKey = data[idx].statusKey; break;
                default: currentKey = 0;
            }
            size_t digit = static_cast<size_t>((currentKey / exp) % BASE);
            output[--count[digit]] = data[idx];
        }
        data = output;
    }
}


void mergeAlphanumeric(vector<SortingData>& data, size_t left, size_t mid,
size_t right) {
    vector<SortingData> temp(right - left + 1);
    size_t i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right) {
        if (data[i].idKey < data[j].idKey) {
            temp[k++] = data[i++];
        } else {
            temp[k++] = data[j++];
        }
    }

    while (i <= mid) temp[k++] = data[i++];
    while (j <= right) temp[k++] = data[j++];

    for (k = 0; k < temp.size(); k++) {
        data[left + k] = temp[k];
    }
}
```

```
void mergeSortAlphanumeric(vector<SortingData>& data, size_t left, size_t
right) {
    if (left < right) {
        size_t mid = left + (right - left) / 2;
        mergeSortAlphanumeric(data, left, mid);
        mergeSortAlphanumeric(data, mid + 1, right);
        mergeAlphanumeric(data, left, mid, right);
    }
}

// ____________________________________ DISPLAY
____________________________________

static string headerBlock, bottomBorder, headerBlock_DiscardPile,
bottomBorder_DiscardPile;

// Brand is not bound by length unlike id{10}, age{3}, and status{7}, which is
why header must (sadly) be dynamic
string headerGen (size_t brandColumnSize, bool isDiscard = false, bool
isBottom = false) {
    string preColumn = (isDiscard) ? " | =======" : "";
    string preColumnText = (isDiscard) ? " |  Index " : "";
    string postColumn = (isDiscard) ? "==================== |" : "";
    string postColumnText = (isDiscard) ? "      Timestamp      |" : "";
    int brandLength = static_cast<int>(brandMaxLength);
    int paddingLeft = (brandLength - 5) / 2;
    int paddingRight = paddingLeft + ((brandLength - 5) % 2);

    ostringstream bdr{};

    // (Im sorry, OCD hit)

    bdr     << preColumn << " | ========== | " << string(brandColumnSize, '=')
<<
" | ========== | ========== | ========== | " << postColumn << "\n";

    if (!isBottom) {
    bdr << preColumnText << " |     ID     | " <<
string(static_cast<size_t>(paddingLeft), ' ') << "Brand" <<
string(static_cast<size_t>(paddingRight), ' ') << " |    Age     |    Date
|   Status   | " << postColumnText << "\n"
```

```
            << preColumn << " | ========== | " << string(brandColumnSize, '=')
<<
" | ========== | ========== | ========== | " << postColumn << "\n";
    }

    return bdr.str();
}

static constexpr string_view invHelpDisp =
"\n================================================================================
================================"
"\n/help Inventory \t#shows slash commands below:"
"\n/add [stock code] [brand] [age] [status] [date] \t# Add new stock \t(alias:
'/a')"
"\n/update age [stock code] [age] \t# Updates available stock's age \t(alias:
'/upd')"
"\n/update brand [stock code] [brand] \t# Updates ... brand \t(alias: '/upd')"
"\n/update date [stock code] [date] \t# Updates ... date \t(alias: '/upd')"
"\n/update status [stock code] [status] \t# Updates ... status \t(alias:
'/upd')"
"\n/remove [stock code] \t# Removes stock and adds it to discard pile
\t(aliases: '/rem', '/del', '/delete')"
"\n/restore list [discard / overwrite] \t# Displays discard/overwrite pile
\t(alias: '/res')"
"\n/restore [discard / overwrite] [index] \t# Adds stock from
discard/overwrite pile to inventory \t(alias: '/res')"
"\n/list all \t# Displays all stocks"
"\n/list filter [brand / age / status / date] [val to filter] \t# Displays all
stocks under that filter"
"\n/sort [id / brand / age / status / date] \t# Displays all stocks (sorted by
specification)"
"\n/[xsearch | fsearch] [stock code] \t# Search (exact or fuzzy) for id and
displays info regarding that id \t(aliases: '/xs', '/fs')"
"\n/mm \t# Return to main menu"
"\n/exit \t# Safely exits the program  \t(aliases: '/quit', '/q', '/x')"
"\n================================================================================
================================";

template<typename T, size_t N>
void writeAndCenter(stringstream& ss, const array<T,N>& temp, bool
nextlineDisable = false) {
    ss << " | ";
```

```
    for (size_t i = 0; i < 5; ++i) {
        string currentString = temp[i];
        size_t spaces = (currentString.length() >= 10) ? 0 : 10 -
currentString.length();
        if ((i == 1) && (brandMaxLength > 10)) spaces = brandMaxLength -
currentString.length();
        ss << string(spaces/2, ' ') << currentString << string((spaces / 2) +
(spaces % 2), ' ') << " | ";
    }
    if (!nextlineDisable) {
        ss << endl;
    }
}

void listFilterType(const invMap& index, const string& filterValue,
stringstream& ss) { // Command: /list filter - sub
    auto range = index.equal_range(caseLower(filterValue));
    vector<const stockData*> entries;
    for (auto i = range.first; i != range.second; ++i) {
        entries.push_back(i->second);
    }

    vector<SortingData> sortingEntries;
    for (const auto* entry : entries) {
        sortingEntries.push_back({
            entry,
            extractDateKey(entry->getDate()),
            brandMap[entry->getBrand()],
            (caseLower(entry->getAge()) == "new" ? 0 : 1),
            (caseLower(entry->getStatus()) == "on-hand" ? 0 : 1),
            entry->getID()
        });
    }

    // Sort by ID first to ensure stability, then by date
    mergeSortAlphanumeric(sortingEntries, 0, sortingEntries.size() - 1);
    radixSort(sortingEntries, DATE_KEY);

    forEach(entry, sortingEntries) {
        array<string, 5> temp = entry.data->getAll();
        writeAndCenter(ss, temp);
    }
```

```
}

#define HeaderFooter(...) do {                \
    ss << headerBlock;                  \
    __VA_ARGS__                         \
    ss << bottomBorder;                 \
} while(0)

void listFilter(const string& filterType, const string& filterValue,
stringstream& ss) { //Comand: list filter
    static unordered_map<string, const invMap&> filterIndices = {
        {"brand", brandIndex}, {"age", ageIndex}, {"date", dateIndex},
{"status", statusIndex}
    };

    ss.str("");
    HeaderFooter({
        onFind(category, filterIndices.find(filterType), filterIndices.end(),
{ // If category returns a invMap from valid key @ filterIndices
            listFilterType(category->second, filterValue, ss); // send map and
value to listFilterType
        });
    });

}

void displaySortedInventory(stringstream& ss, const vector<SortingData>&
sortedData) {
    HeaderFooter({
        forEach(item, sortedData) {
            array<string, 5> data = item.data->getAll();
            writeAndCenter(ss, data);
        }
    });
}
// ____________________________________ DISPLAY CACHE
____________________________________

stringstream dispList, dispList_AgeNew, dispList_AgeOld, dispList_StatusOH,
dispList_StatusSd,
    dispSort_ID, dispSort_Brand, dispSort_Age, dispSort_Date, dispSort_Status;
```

```
mutex dispListMutex, dispList_AgeNewMutex, dispList_AgeOldMutex,
dispList_StatusOHMutex, dispList_StatusSdMutex,
    dispSort_IDMutex, dispSort_BrandMutex, dispSort_AgeMutex,
dispSort_DateMutex, dispSort_StatusMutex;


void trigUpdate() {
    clog << "Cache update triggered. Starting Caching..." << endl;
    auto tStart = Clock::now();
    buildBrandMap();
    headerBlock = headerGen(brandMaxLength);
    bottomBorder = headerGen(brandMaxLength,0,1);
    headerBlock_DiscardPile = headerGen(brandMaxLength,1,0);
    bottomBorder_DiscardPile = headerGen(brandMaxLength,1,1);
    prepareSortingData();

    vector<thread> threads;

    // ListAll
    threads.emplace_back([&]() {
            lock_guard<mutex> invLock(inventoryMutex);
            vector<SortingData> local = sortedInventory;
            sort(local.begin(), local.end(), [](const SortingData& a, const
SortingData& b) {
                if (a.dateKey != b.dateKey) {
                    return a.dateKey < b.dateKey;
                } else {
                    return a.idKey < b.idKey;
                }
            });
            stringstream ss;
            displaySortedInventory(ss, local);
            lock_guard<mutex> lock(dispListMutex);
            dispList.str(ss.str());
        });

    // Corrected addUpdThread macro definition
    #define addUpdThread(filterType, filterValue, mx, stream) \
        threads.emplace_back([&]() { \
            stringstream ss; \
            listFilter(filterType, filterValue, ss); \
            lock_guard<mutex> lock(mx); \
```

```
            stream.str(ss.str()); \
        })

    // Age filters
    addUpdThread("age", "new", dispList_AgeNewMutex, dispList_AgeNew);
    addUpdThread("age", "old", dispList_AgeOldMutex, dispList_AgeOld);

    // Status filter
    addUpdThread("status", "on-hand", dispList_StatusOHMutex,
dispList_StatusOH);
    addUpdThread("status", "sold", dispList_StatusSdMutex, dispList_StatusSd);

   // Sorted lists
    #define sortCache(primaryKey, secondaryKey, mtx, outputStream) \
    threads.emplace_back([&]() { \
        vector<SortingData> local = sortedInventory; \
        if (primaryKey == ID_KEY) mergeSortAlphanumeric(local, 0, local.size()
- 1); \
        radixSort(local, DATE_KEY); \
        if (secondaryKey < ID_KEY) { \
            radixSort(local, secondaryKey); \
        } else if (secondaryKey != DATE_KEY) { \
            mergeSortAlphanumeric(local, 0, local.size() - 1); \
        } \
        stringstream ss; \
        displaySortedInventory(ss, local); \
        lock_guard<mutex> lock(mtx); \
        outputStream.str(ss.str()); \
    });

    sortCache(ID_KEY, DATE_KEY, dispSort_DateMutex, dispSort_Date)
    sortCache(DATE_KEY, BRAND_KEY, dispSort_BrandMutex, dispSort_Brand)
    sortCache(DATE_KEY, AGE_KEY, dispSort_AgeMutex, dispSort_Age)
    sortCache(DATE_KEY, STATUS_KEY, dispSort_StatusMutex, dispSort_Status)
    sortCache(DATE_KEY, ID_KEY, dispSort_IDMutex, dispSort_ID)

    // *cede the idea to combine filter age (old & new) and filter status
(on-hand & sold) into one cache each for linearity
    // (yes, I chose practicality and readability over tc and sc)

    for (auto& t : threads) if (t.joinable()) t.join();
```

```
    lock_guard<mutex> coutLock(clockElapsed);
    auto duration = chrono::duration<double, milli>(Clock::now() -
tStart).count();
    clog << "Cache load completed in "
         << duration << " ms.\n";
}
// ____________________________________ INVENTORY COMMANDS
_____________________________________

#define grabTokens const vector<string> &tokens

vector<pair<string, int>> performFuzzySearch(const string& searchTerm, int
threshold) {
    vector<pair<string, int>> matches;
    matches.reserve(inventory.size() / 4);
    lock_guard<mutex> lock(inventoryMutex);
    forEach(item, inventory) {
        int distance = levDist(searchTerm, item.first, threshold);
        if (distance <= threshold) matches.emplace_back(item.first, distance);
        if (matches.size() == matches.capacity())
matches.reserve(matches.capacity() * 2);
    }
    sort(matches.begin(), matches.end(), [](const pair<string, int>& a, const
pair<string, int>& b) {
        return a.second < b.second;
    });
    return matches;
}

// Overloaded function to validate fields simpler
bool validateFields(const string& id, const string& date, const string& age,
const string& status) {
    if (!validID(id)) { errInv(IN_ID, "Invalid ID: " + id); return false; }
    if (!validDate(date)) { errInv(IN_DATE, "Invalid date: " + date); return
false; }
    if (!validAge(age)) { errInv(IN_AGE, "Invalid age: " + age); return false;
}
    if (!validStatus(status)) { errInv(IN_STATUS, "Invalid status: " +
status); return false; }
    return true;
}
```

```
stack<string> commandLog;
mutex commandLogMutex;

void commandHistory() {
    time_t exitTime = currentTime;
    tm* tmInfo = localtime(&exitTime);
    char timestamp[20];
    strftime(timestamp, sizeof(timestamp), "%Y-%m-%d %H:%M:%S", tmInfo);

    //get existing logs
    vector<string> existingLogs;
    ifstream inLog("command_log.txt");
    if(!inLog.is_open()) { errInv(FILE_LOAD, "command_log.txt"); return; }
    string line;
    while (getline(inLog, line)) {
        existingLogs.push_back(line);
    }
    inLog.close();

    //write current log to existing logs ofstream outLog("command_log.txt");
    ofstream outLog("command_log.txt");
    outLog << "\n" <<timestamp << "\n";
    {
        lock_guard<mutex> lock(commandLogMutex);
        while (!commandLog.empty()) { outLog << commandLog.top() << "\n";
commandLog.pop(); }
    }
    forEach(existingLines, existingLogs) {
        outLog << existingLines << "\n";
    }
}

void quit() {
    saveDataCSV("inventory.csv", inventory);
    saveDataCSV("discards.csv", discards);
    saveDataCSV("overwriteHistory.csv", overwriteHistory);
    commandHistory();
    exit(0);
}

enum tokenSizeSpecification { mustEqual, aroundEqual };
```

```
inline bool tokenSizeCheck(const string_view& usage, const vector<string>&
tokens, tokenSizeSpecification spec, size_t size) {
    if (spec == mustEqual && tokens.size() != size) {
        errInv(INVALID_COMMAND, usage.data()); return true;
    } else if (spec == aroundEqual && tokens.size() < size) {
        errInv(INVALID_COMMAND, usage.data()); return true;
    }
    return false;
}

string fuzzyMatchID(const string& inputID) {
    if (validID(inputID)) return inputID; // Return valid IDs immediately
    if (inputID.length() >= 10) return ""; // Not eligible for fuzzy search

    static constexpr int threshold = 7;
    vector<pair<string, int>> matches;

    { // lock to protect inventory access
        lock_guard<mutex> lock(inventoryMutex);
        forEach(item, inventory) {
            int distance = levDist(inputID, item.first, threshold);
            if (distance <= threshold) matches.emplace_back(item.first,
distance);
        }
    }

    if (matches.empty()) return ""; //no matches found

    //sort matches by distance and return closest match
    sort(matches.begin(), matches.end(), [](const auto& a, const auto& b) {
        return a.second < b.second;
    });

    return matches.front().first;
}

string tryFuzzy (string const& id) {
    clog << "Stock ID not found: " << id << endl
        << "Trying fuzzy search for ID: " << id << endl;
    string matchedID = fuzzyMatchID(id);
    if (matchedID.empty()) {
        errInv(IN_ID, "Invalid ID (Fuzzy Failed): " + id);
```

```
        return "";
    }
    return matchedID;
}

inline auto findStock(string &id) -> decltype(inventory.begin()) {
    auto it = inventory.find(id);
    if (it == inventory.end()) {
        id = tryFuzzy(id);
        if (id.empty()) return inventory.end();
        it = inventory.find(id);
    }
    return it;
}

void commandInvHelp(grabTokens) {
    if (tokens.size() == 1) cout << "Command usage: /help
[Inventory/inv/Payroll/pr]\n";
    else if ((tokens[1] == "inventory") || (tokens[1] == "inv")) cout <<
invHelpDisp << endl;
    else if ((tokens[1] == "payroll") || (tokens[1] == "pr")) cout << "Payroll
part of the program isn't finished yet" << endl;
    else errInv(INVALID_COMMAND, "Command usage: /help
[Inventory/inv/Payroll/pr]");
}

void commandAdd(grabTokens) {
    if (tokenSizeCheck("/add [stock code] [brand] [age] [status] [date]",
tokens, mustEqual, 6)) return;

    string id = caseUp(tokens[1]), brand = caseCap(tokens[2]), age =
caseCap(tokens[3]), status = caseCap(tokens[4]), date = tokens[5];

    if (!validateFields(id, date, age, status)) return;

    {
        // Lock all shared data
        lock_guard<mutex> invLock(inventoryMutex);
        onFind(existing, inventory.find(id),inventory.end(), {

            if (!confirmAction("overwrite", id)) {
                clog << "Operation cancelled." << endl;
```

```
                return;
            }

            stockData& oldData = existing->second;
            removeFromIndexes(oldData, true);

            // Add to overwriteHistory and erase from inventory
            overwriteHistory.emplace_front(oldData, currentTime);
            if (overwriteHistory.size() > 16) overwriteHistory.pop_back();
            inventory.erase(existing);
        });
        stockData newData(id, brand, age, date, status);
        auto& insertedData = inventory[id]; // Insert or assign the new data
        insertedData = newData;
        addToIndexes(&insertedData);

        clog << "Successfully added Stock Instance: " << id << endl;
    }
    trigUpdate(); // Ensure trigUpdate is called after releasing the lock if
necessary
}

void commandUpdate(grabTokens) {
    if (tokenSizeCheck("/update [age/brand/date/status] [stock code] [value]",
tokens, mustEqual, 4)) return;

    string field = tokens[1], id = caseUp(tokens[2]), newVal = (field !=
"brand") ? caseCap(tokens[3]) : tokens[3];
    newVal = caseCap(newVal);

    auto it = findStock(id);
    if (it == inventory.end()) return;

    cout << "Updating stock: " << id << endl;

    stockData& data = it->second;
    string oldBrand = data.getBrand();
    string oldAge = data.getAge();
    string oldDate = data.getDate();
    string oldStatus = data.getStatus();
```

```
    if (!confirmAction("update", id)) {
        clog << "Operation cancelled." << endl;
        return;
    }

    if (field == "age") {
        if (!validAge(newVal)) {
            errInv(IN_AGE, "Invalid age: " + newVal);
            return;
        }
        data.setAge(newVal);
    } else if (field == "brand") {
        data.setBrand(newVal);
    } else if (field == "date") {
        if (!validDate(newVal)) {
            errInv(IN_DATE, "Invalid date: " + newVal);
            return;
        }
        data.setDate(newVal);
    } else if (field == "status") {
        if (!validStatus(newVal)) {
            errInv(IN_STATUS, "Invalid status: " + newVal);
            return;
        }
        data.setStatus(newVal);
    } else {
        errInv(INVALID_COMMAND, "Invalid update field: " + field);
        return;
    }

    overwriteHistory.emplace_front(data, currentTime);
    if (overwriteHistory.size() > 16) overwriteHistory.pop_back();

    removeFromIndexes(data);
    addToIndexes(&data);
    trigUpdate();
}

void commandRemove(grabTokens) {
    if (tokenSizeCheck("/remove [stock code]", tokens, mustEqual, 2)) return;

    string id = caseUp(tokens[1]);
```

```
    auto it = findStock(id);
    if (it == inventory.end()) return;

    if (!confirmAction("discard", id)) {
        clog << "Operation cancelled." << endl;
        return;
    }

    {
        lock_guard<mutex> lock(inventoryMutex);
        discards.emplace_front(it->second, currentTime);
        if (discards.size() > 16) discards.pop_back();

        removeFromIndexes(it->second);
        inventory.erase(it);
    }
    trigUpdate();
}

void commandRestore(grabTokens) {
    if (tokenSizeCheck("/restore list [discard/overwrite] or /restore
[discard/overwrite] [index]", tokens, aroundEqual, 2)) return;

    if (tokens[1] == "list") {
        if (tokenSizeCheck("/restore list [discard/overwrite]", tokens,
mustEqual, 3)) return;

        const deque<discardEntry>* pile = (tokens[2] == "discard") ? &discards
: &overwriteHistory;
        stringstream ss;
        ss << headerBlock_DiscardPile;
        for (size_t i = 0; i < pile->size(); ++i) {
            const auto& entry = (*pile)[i];
            ss << " | " << string((i > 9) ? 5 : 6, ' ') << i;
            writeAndCenter(ss, entry.data.getAll(), true);
            ss << " " << put_time(localtime(&entry.timestamp), "%Y-%m-%d
%H:%M:%S") << " |\n";
        }
        ss << bottomBorder_DiscardPile;
        cout << ss.str();
    } else {
```

```
        if (tokenSizeCheck("/restore [discard/overwrite] [index]", tokens,
mustEqual, 3)) return;

        int index;
        try { index = stoi(tokens[2]); }
        catch (...) { errInv(INVALID_COMMAND, "Invalid index: " + tokens[2]);
return; }

        deque<discardEntry>* pile = (tokens[1] == "discard") ? &discards :
&overwriteHistory;
        if (index < 0 || static_cast<size_t>(index) >= pile->size()) {
            errInv(INVALID_COMMAND, "Index out of range: " +
to_string(index));
            return;
        }

        auto entry = (*pile)[static_cast<size_t>(index)];
        {
            lock_guard<mutex> lock(inventoryMutex);

            onFind(existing, inventory.find(entry.data.getID()),
inventory.end(), {
                if (!confirmAction("overwrite", entry.data.getID())) {
                    clog << "Operation cancelled." << endl;
                    return;
                }

                overwriteHistory.push_front(discardEntry(existing->second,
currentTime));
                if (overwriteHistory.size() > 16) overwriteHistory.pop_back();
                inventory.erase(existing);
            });

            inventory[entry.data.getID()] = entry.data;
            stockData* ptr = &inventory[entry.data.getID()];
            addToIndexes(ptr);
            pile->erase(pile->begin() + index);
        }
        trigUpdate();
    }
}
```

```
void commandList(grabTokens) {
    if (tokens.size() < 2) {
        errInv(INVALID_COMMAND, "Usage: /list [all/filter]");
        return;
    }

    if (tokens[1] == "all") {
        lock_guard<mutex> lock(dispListMutex);
        cout << dispList.str();
    } else if (tokens[1] == "filter" && tokens.size() == 4) {
        string filterType = tokens[2], filterValue = tokens[3];
        stringstream ss;
        if (filterType == "brand") listFilterType(brandIndex, filterValue,
ss);
        else if (filterType == "age") listFilterType(ageIndex, filterValue,
ss);
        else if (filterType == "status") listFilterType(statusIndex,
filterValue, ss);
        else if (filterType == "date") listFilterType(dateIndex, filterValue,
ss);
        else { errInv(FILTER_NOTFOUND, "Invalid filter type: " + filterType);
return; }
        cout << headerBlock << ss.str() << bottomBorder;
    } else {
        errInv(INVALID_COMMAND, "Invalid subcommand for /list");
    }
}

void commandSort(grabTokens) {
    if (tokens.size() != 2) {
        errInv(INVALID_COMMAND, "Usage: /sort [id/brand/age/status/date]");
        return;
    }

    string key = tokens[1];
    if (key == "id") { lock_guard<mutex> lock(dispSort_IDMutex); cout <<
dispSort_ID.str(); }
    else if (key == "brand") { lock_guard<mutex> lock(dispSort_BrandMutex);
cout << dispSort_Brand.str(); }
    else if (key == "age") { lock_guard<mutex> lock(dispSort_AgeMutex); cout
<< dispSort_Age.str(); }
```

```
    else if (key == "status") { lock_guard<mutex> lock(dispSort_StatusMutex);
cout << dispSort_Status.str(); }
    else if (key == "date") { lock_guard<mutex> lock(dispSort_DateMutex); cout
<< dispSort_Date.str(); }
    else errInv(INVALID_COMMAND, "Invalid sort key: " + key);
}

void commandSearch(grabTokens) {
    if (tokens.size() != 2) {
        errInv(INVALID_COMMAND, "Usage: /xsearch [stock code]");
        return;
    }

    string id = caseUp(tokens[1]);
    if (!validID(id)) { errInv(IN_ID, "Invalid ID: " + id); return; }

    lock_guard<mutex> lock(inventoryMutex);
    auto it = inventory.find(id);
    if (it == inventory.end()) {
        cout << "Stock ID not found: " << id << endl;
        return;
    }

    array<string, 5> data = it->second.getAll();
    stringstream ss;
    HeaderFooter({writeAndCenter(ss, data);});
    cout << ss.str();
}

void commandFuzzySearch(grabTokens) {
    if (tokens.size() != 2) {
        errInv(INVALID_COMMAND, "Usage: /fsearch [stock code]");
        return;
    }

    string searchTerm = caseUp(tokens[1]);
    static constexpr int threshold = 7;  //must have at least 3 characters to
match
    vector<pair<string, int>> matches = performFuzzySearch(searchTerm,
threshold);

    if (matches.empty()) {
```

```
        cout << "No matches found for: " << searchTerm << endl;
        return;
    }

    sort(matches.begin(), matches.end(), [](const pair<string, int>& a, const
pair<string, int>& b) {
        return a.second < b.second;
    });

    cout << "Fuzzy search results for: " << searchTerm << endl;
    stringstream ss;
    HeaderFooter({
        forEach(match, matches) {
            onFind(it, inventory.find(match.first), inventory.end(), {
                array<string, 5> data = it->second.getAll();
                writeAndCenter(ss, data);
            });
        }
    });
    cout << ss.str();
}

void processInventoryCommands() {
    system("pause");
    system("cls");
    string command;
    cout << "Please input a command to continue (use '/help Inventory' for
assistance).\n";

    unordered_map<string, function<void(const vector<string>&)>> commandMap =
{
        {"/help", commandInvHelp},
        {"/add", commandAdd},
        {"/upd", commandUpdate},
        {"/update", commandUpdate},
        {"/rem", commandRemove},
        {"/del", commandRemove},
        {"/remove", commandRemove},
        {"/delete", commandRemove},
        {"/res", commandRestore},
        {"/restore", commandRestore},
        {"/list", commandList},
```

```
        {"/sort", commandSort},
        {"/xsearch", commandSearch},
        {"/xs", commandSearch},
        {"/fsearch", commandFuzzySearch},
        {"/fs", commandFuzzySearch},
        {"/x", [](const vector<string>&) { quit(); }},
        {"/exit", [](const vector<string>&) { quit(); }},
        {"/q", [](const vector<string>&) { quit(); }},
        {"/quit", [](const vector<string>&) { quit(); }}
    };

    while (true) {
        cout << "> ";
        getline(cin, command);
        vector<string> tokens = splitCommand(caseLower(command));
        if (tokens.empty()) continue; // Request for input again

        {
            lock_guard<mutex> lock(commandLogMutex);
            commandLog.push(command);
        }

        command = tokens[0];

        if (command == "/mm") return;
        if (auto it = commandMap.find(command); it != commandMap.end()) {
            it->second(tokens);
        } else {
            errInv(INVALID_COMMAND, command);
        }
    }
}

bool alreadyExtracted = false;
void mainMenu() {
    string input;
    while (true) {
        cout << "Main menu:\n[1] Inventory\n[2] Payroll\n[3] Exit\n";
        cout << "Enter choice: ";
        getline(cin, input);
        input = caseLower(input);
```

```
        if (input == "1" || input == "i" || input == "inv" || input ==
"inventory") {
            if (!alreadyExtracted) { mainExtraction(); alreadyExtracted =
true; }
            processInventoryCommands();
        } else if (input == "2" || input == "p" || input == "pr" || input ==
"payroll") {
            cout << "This part of the program is not yet done.\n";
        } else if (input == "3" || input == "exit") {
            quit();
        } else {
            cout << "Invalid choice. Please try again.\n";
        }
    }
}

// ____________________________________________ MAIN
_____________________________________________

int main() {
    system("pause");
    mainMenu();
    return 0;
}
```